Dynamic Programming and Search Trees

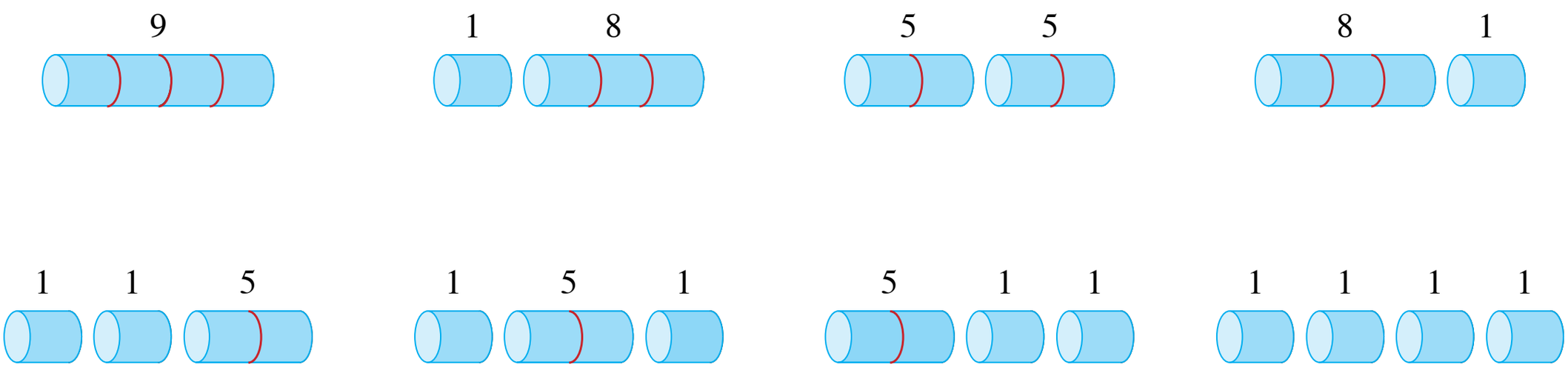
Last Lecture for course in Data Structures & Algorithms for AI

Lectures by Hannah Santa Cruz Baur.
Based on *Introduction to Algorithms*, Cormen et al.

Rod Cutting Problem

Given selling prices for different rod lengths:

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

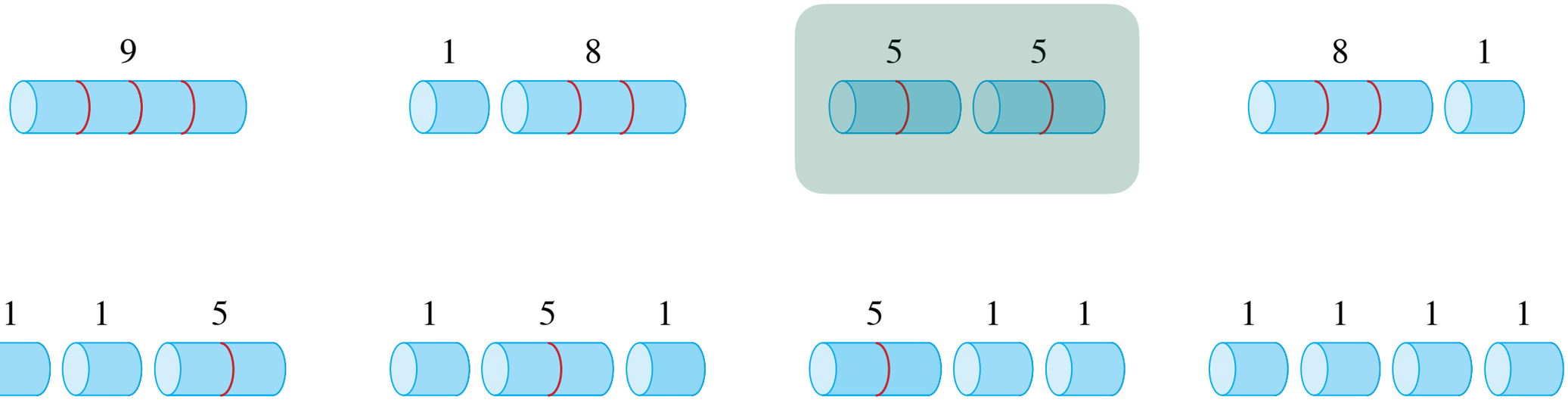


We want to maximize the revenue obtainable by cutting the rod and selling the pieces.

Rod Cutting Problem

Given selling prices for different rod lengths:

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30



We want to maximize the revenue obtainable by cutting the rod and selling the pieces.

Rod Cutting Problem

Given selling prices for different rod lengths, we want to maximize the revenue obtainable by cutting the rod and selling the pieces.

In general, we may formalize the problem as seeking:

$$r_n = max\{p_i + r_{n-i} : 1 \leq i \leq n\}$$

Rod Cutting Problem

Given selling prices for different rod lengths, we want to maximize the revenue obtainable by cutting the rod and selling the pieces.

In general, we may formalize the problem as seeking:

$$r_n = max\{p_i + r_{n-i} : 1 \leq i \leq n\}$$

r_n: maximum revenue obtainable from a rod of length n

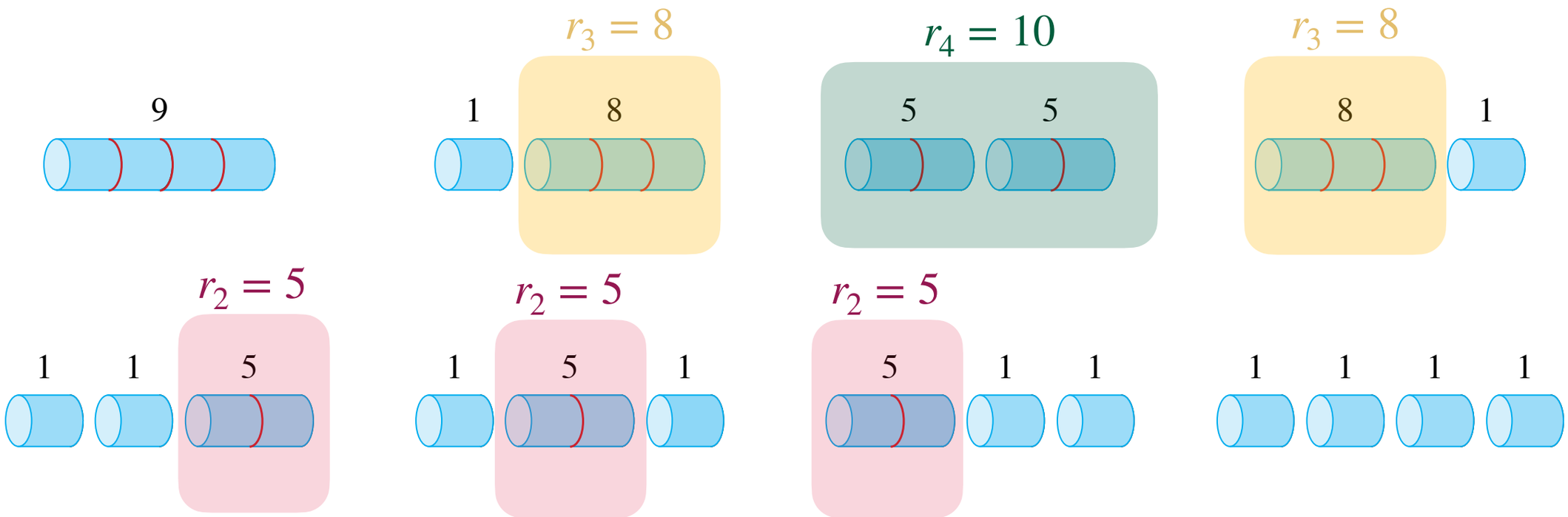
p_i: price of piece of length i

Rod Cutting Problem

Given selling prices for different rod lengths, we want to maximize the revenue obtainable by cutting the rod and selling the pieces.

In general, we may formalize the problem as seeking:

$$r_n = max\{p_i + r_{n-i} : 1 \leq i \leq n\}$$



Rod Cutting Problem

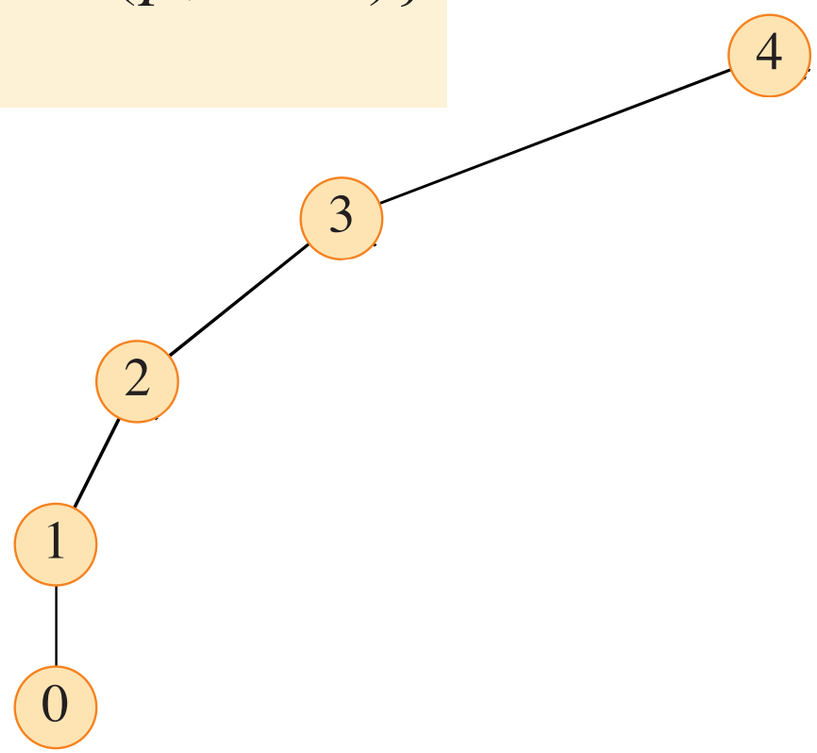
Given selling prices for different rod lengths, we want to maximize the revenue obtainable by cutting the rod and selling the pieces.

Recursive solution:

```
CUT-ROD(p, n)   p = input array of prices
1   if n == 0
2         return 0
3   q = −∞
4   for i = 1 to n
5         q = max{q, p[i] + CUT-ROD(p, n − i)}
6   return q
```

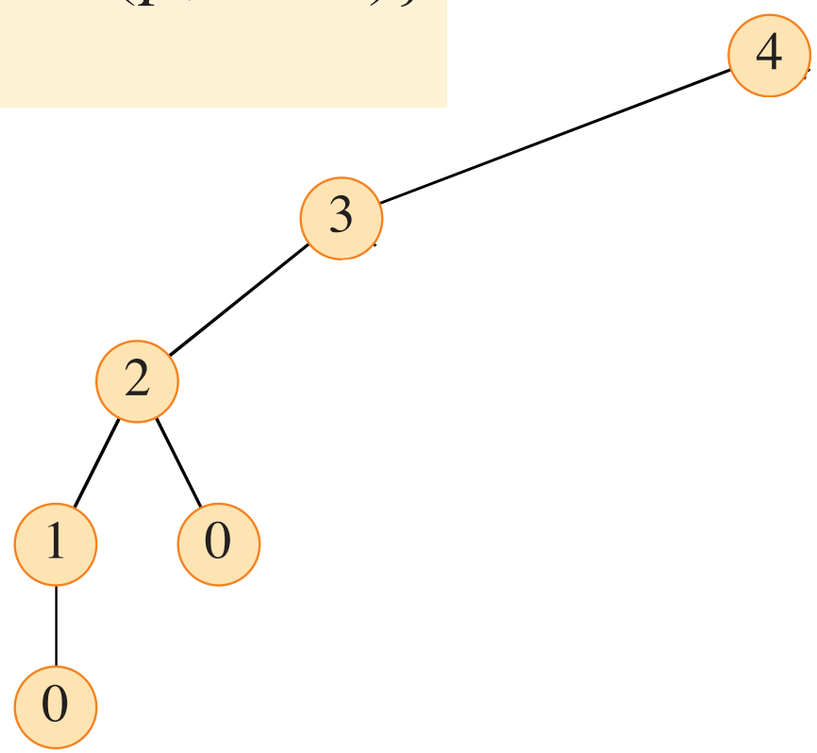
Rod Cutting Problem

```
CUT-ROD(p, n)
1   if n == 0
2         return 0
3   q = −∞
4   for i = 1 to n
5         q = max{q, p[i] + CUT-ROD(p, n − i)}
6   return q
```



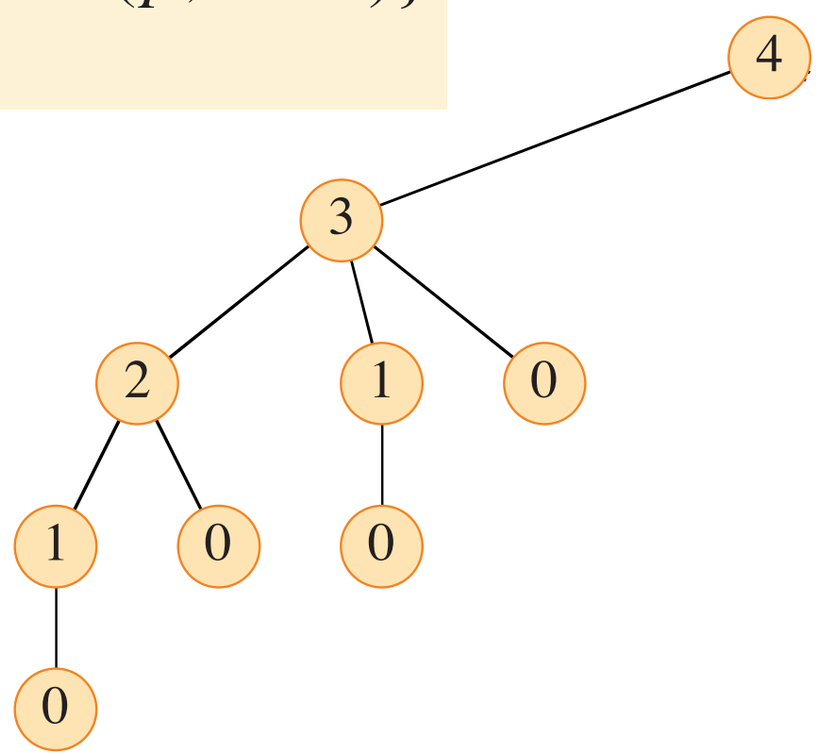
Rod Cutting Problem

```
CUT-ROD(p, n)
1   if n == 0
2         return 0
3   q = −∞
4   for i = 1 to n
5         q = max{q, p[i] + CUT-ROD(p, n − i)}
6   return q
```



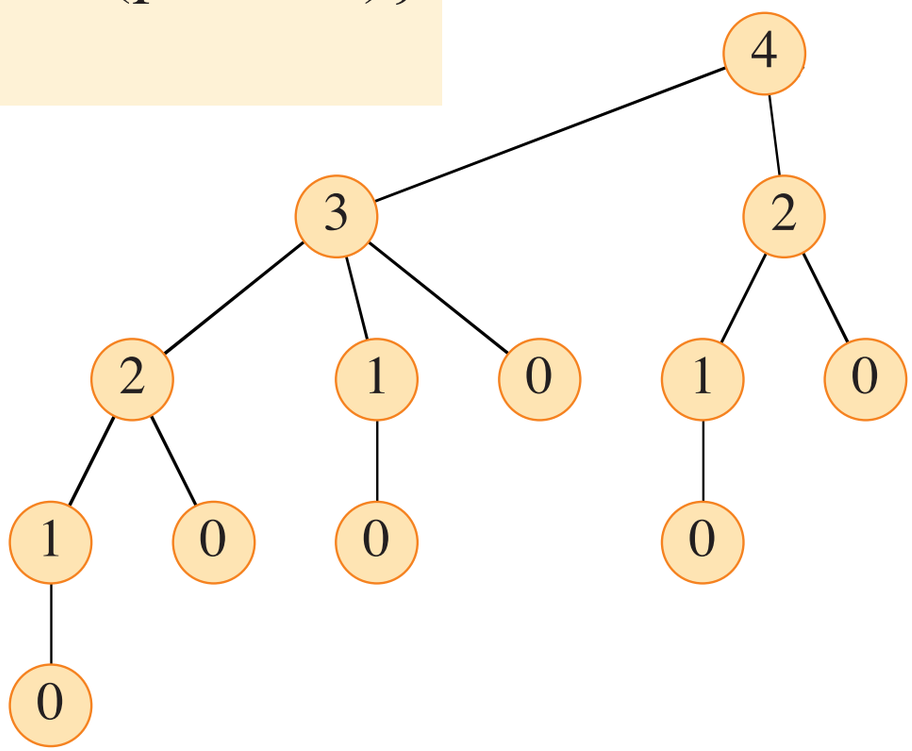
Rod Cutting Problem

```
CUT-ROD(p, n)
1   if n == 0
2         return 0
3   q = −∞
4   for i = 1 to n
5         q = max{q, p[i] + CUT-ROD(p, n − i)}
6   return q
```



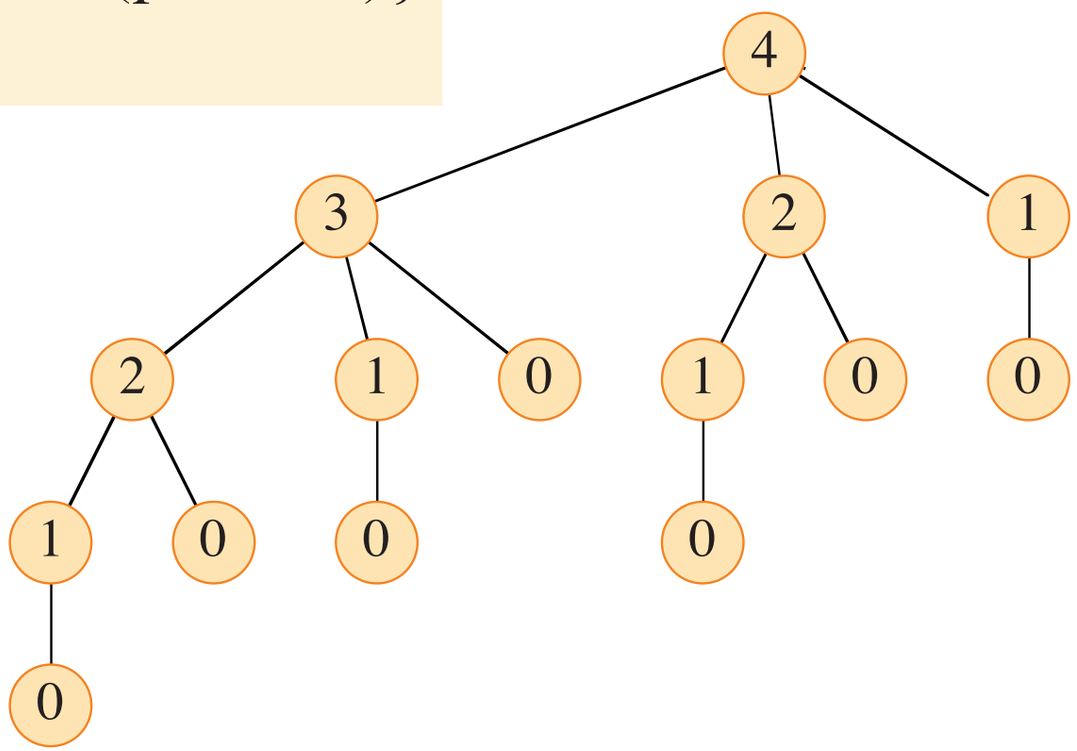
Rod Cutting Problem

```
CUT-ROD(p, n)
1   if n == 0
2         return 0
3   q = −∞
4   for i = 1 to n
5         q = max{q, p[i] + CUT-ROD(p, n − i)}
6   return q
```



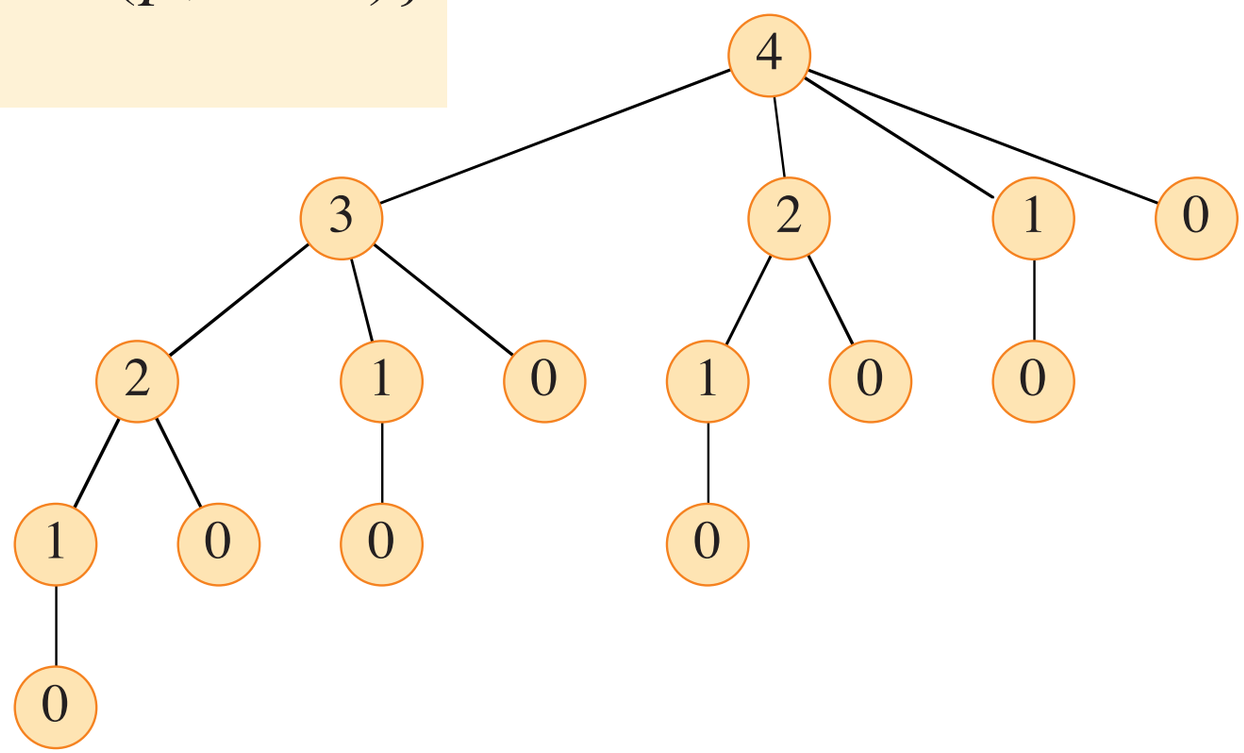
Rod Cutting Problem

```
CUT-ROD(p, n)
1  if n == 0
2        return 0
3  q = −∞
4  for i = 1 to n
5        q = max{q, p[i] + CUT-ROD(p, n − i)}
6  return q
```



Rod Cutting Problem

```
CUT-ROD(p, n)
1  if n == 0
2        return 0
3  q = −∞
4  for i = 1 to n
5        q = max{q, p[i] + CUT-ROD(p, n − i)}
6  return q
```



Will solve the same problem over and over …

Rod Cutting Problem

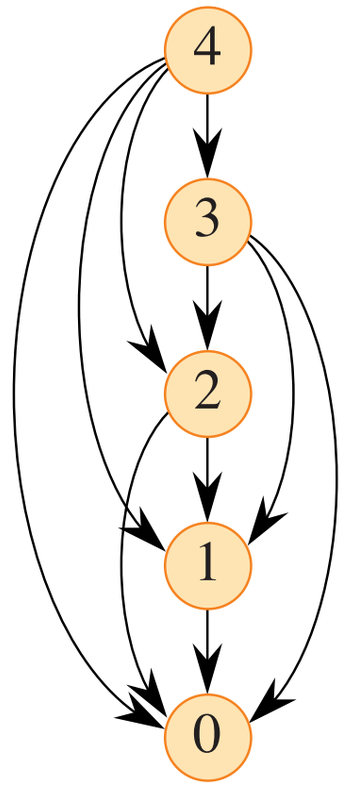
Dynamic programming solution:

```
BOTTOM-UP-CUT-ROD(p, n)
1  let r[0 : n] be a new array          // will remember solution values in r
2  r[0] = 0
3  for j = 1 to n                       // for increasing rod length j
4      q = −∞
5      for i = 1 to j                   // i is the position of the first cut
6          q = max{q, p[i] + r[j − i]}
7      r[j] = q                         // remember the solution value for length j
8  return r[n]
```

Rod Cutting Problem

Dynamic programming solution:

```
BOTTOM-UP-CUT-ROD(p, n)
1  let r[0 : n] be a new array
2  r[0] = 0
3  for j = 1 to n
4       q = −∞
5       for i = 1 to j
6            q = max{q, p[i] + r[j − i]}
7       r[j] = q
8  return r[n]
```



We consider the vertices in **reverse topological sort** order.

Rod Cutting Problem

To keep track of the cuts to be made, add lines:

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
 1  let r[0 : n] and s[1 : n] be new arrays
 2  r[0] = 0
 3  for j = 1 to n                    // for increasing rod length j
 4      q = −∞
 5      for i = 1 to j                // i is the position of the first cut
 6          if q < p[i] + r[j − i]
 7              q = p[i] + r[j − i]
 8              s[j] = i              // best cut location so far for length j
 9      r[j] = q                      // remember the solution value for length j
10  return r and s
```

Rod Cutting Problem

To keep track of the cuts to be made, add lines:

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
 1  let r[0 : n] and s[1 : n] be new arrays
 2  r[0] = 0
 3  for j = 1 to n
 4      q = −∞
 5      for i = 1 to j
 6          if q < p[i] + r[j − i]
 7              q = p[i] + r[j − i]
 8              s[j] = i
 9      r[j] = q
10  return r and s
```

```
PRINT-CUT-ROD-SOLUTION(p, n)
1  (r, s) = EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
2  while n > 0
3      print s[n]
4      n = n − s[n]
```

Rod Cutting Problem

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
 1  let r[0:n] and s[1:n] be new arrays
 2  r[0] = 0
 3  for j = 1 to n
 4      q = -∞
 5      for i = 1 to j
 6          if q < p[i] + r[j - i]
 7              q = p[i] + r[j - i]
 8              s[j] = i
 9      r[j] = q
10  return r and s
```

```
PRINT-CUT-ROD-SOLUTION(p, n)
1  (r, s) = EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
2  while n > 0
3      print s[n]
4      n = n - s[n]
```

PRINT-CUT-ROD-SOLUTION(p, 7)

Rod Cutting Problem

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
 1  let r[0:n] and s[1:n] be new arrays
 2  r[0] = 0
 3  for j = 1 to n
 4      q = -∞
 5      for i = 1 to j
 6          if q < p[i] + r[j - i]
 7              q = p[i] + r[j - i]
 8              s[j] = i
 9      r[j] = q
10  return r and s
```

```
PRINT-CUT-ROD-SOLUTION(p, n)
 1  (r, s) = EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
 2  while n > 0
 3      print s[n]
 4      n = n - s[n]
```

PRINT-CUT-ROD-SOLUTION(p, 10)

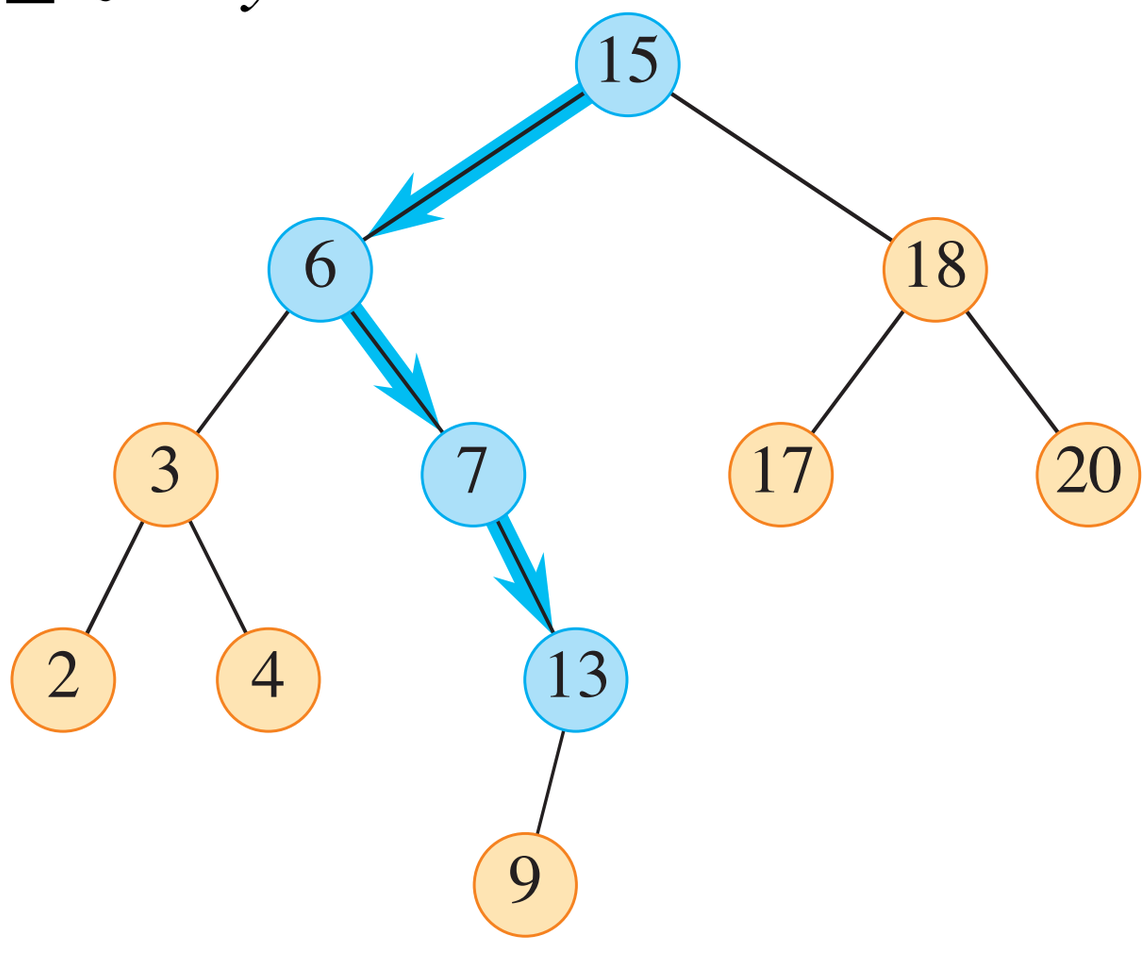
10

PRINT-CUT-ROD-SOLUTION(p, 7)

1,6

Binary Search Trees

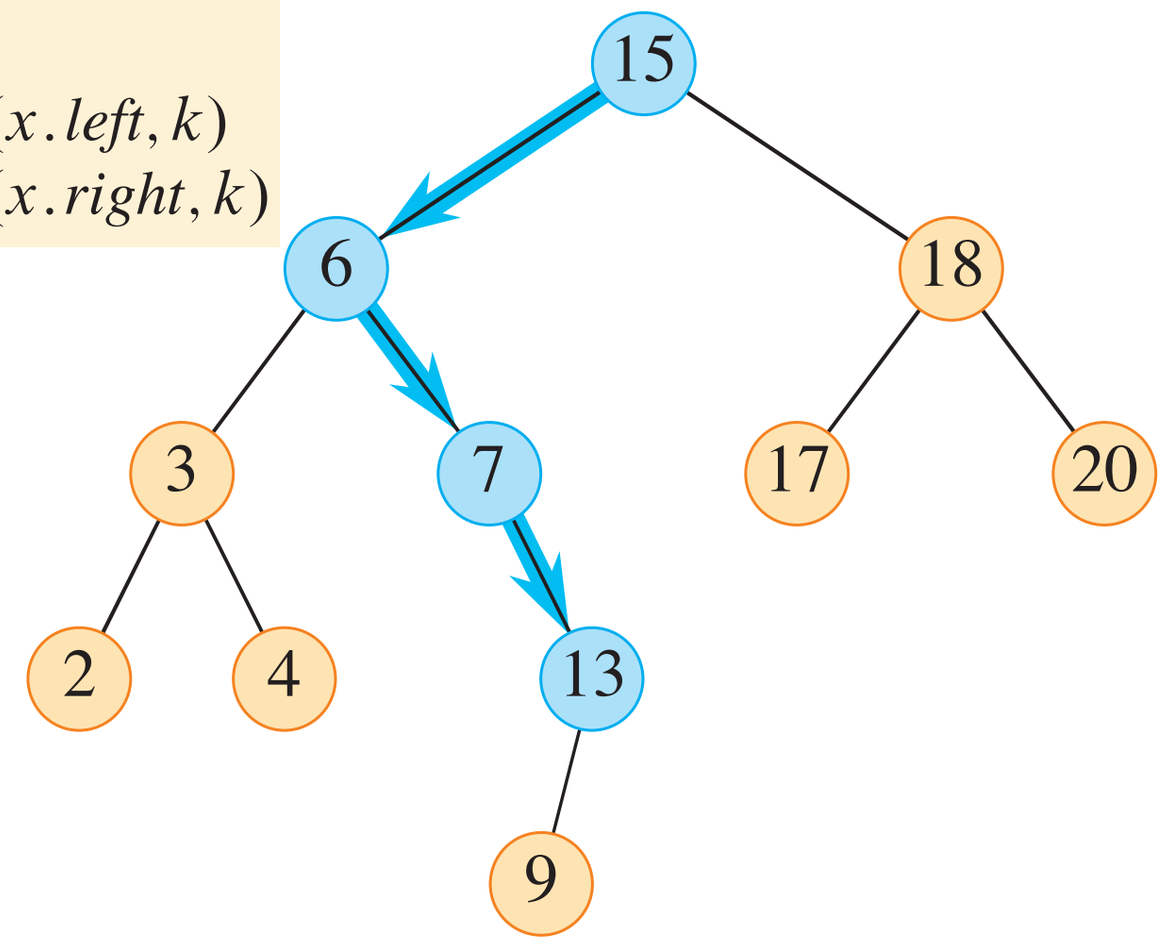
- A **Binary Search Tree** is a data structure which can be viewed as a binary tree, satisfying a key property:
- For any node x, and nodes y and z in it's left and right subtrees respectively, it holds that:
- $y\,.\,key \leq x\,.\,key \leq z\,.\,key$



Binary Search Trees

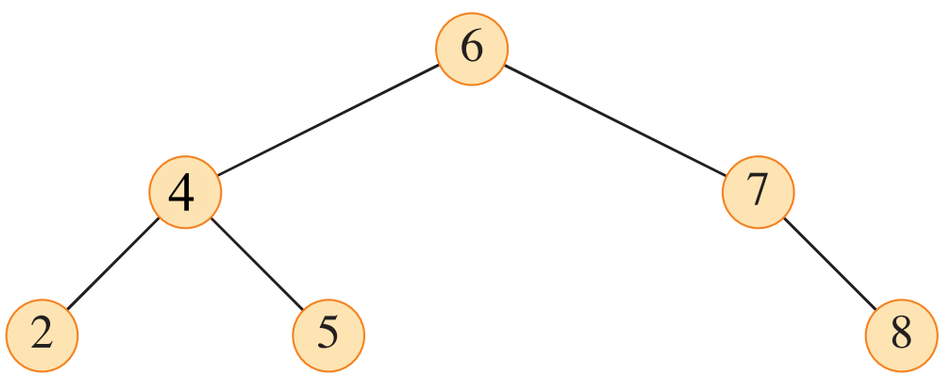
- The *binary search tree property* enables for an efficient search algorithm.

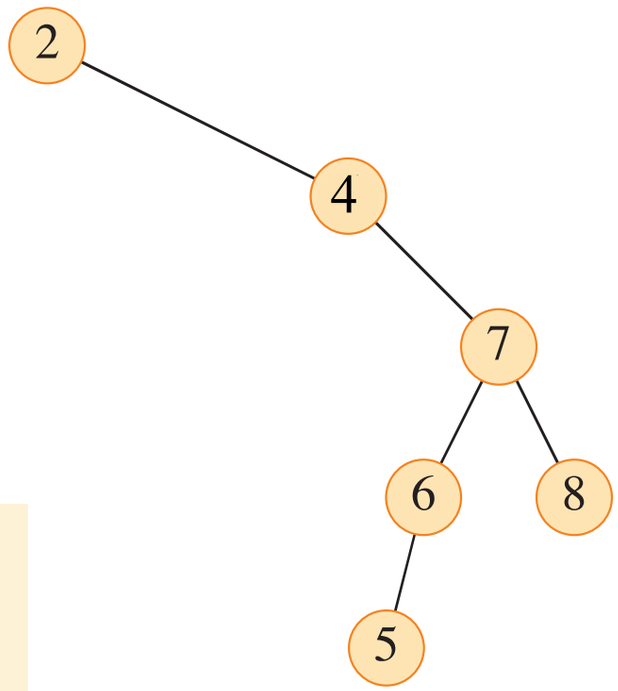
```
TREE-SEARCH(x, k)
1   if x == NIL or k == x.key
2         return x
3   if k < x.key
4         return TREE-SEARCH(x.left, k)
5   else return TREE-SEARCH(x.right, k)
```



Binary Search Trees

How "balanced" a tree is however, can have a large impact on search time. Try SEARCH(root,5) for each tree.

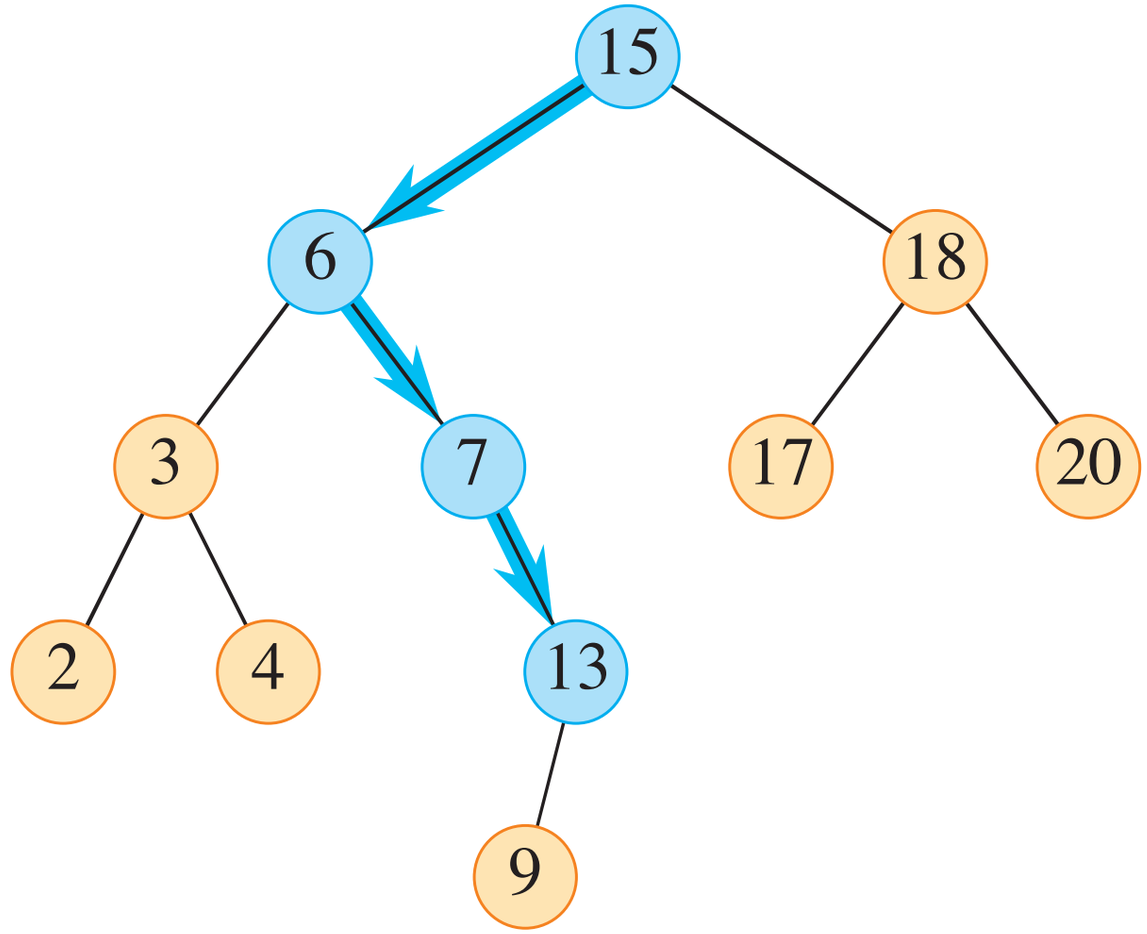





```
TREE-SEARCH(x, k)
1   if x == NIL or k == x.key
2         return x
3   if k < x.key
4         return TREE-SEARCH(x.left, k)
5   else return TREE-SEARCH(x.right, k)
```

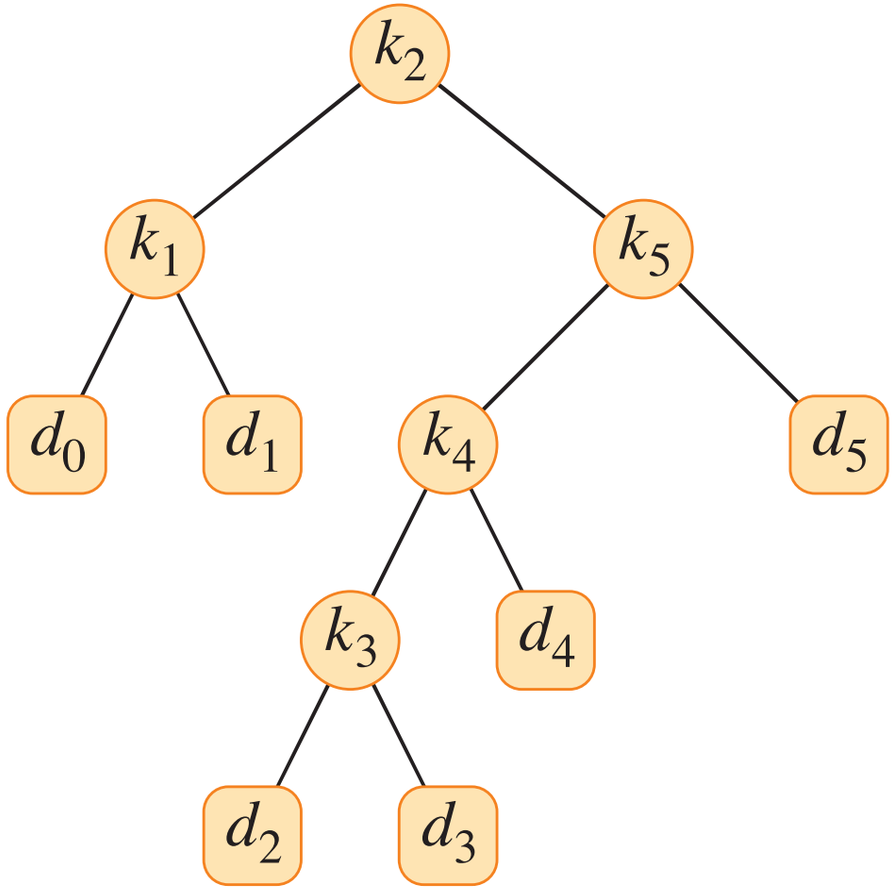
Binary Search Trees

- When contemplating the use of a binary search tree a repeated number of times, we must also consider whether some keys will be searched for more often than others.



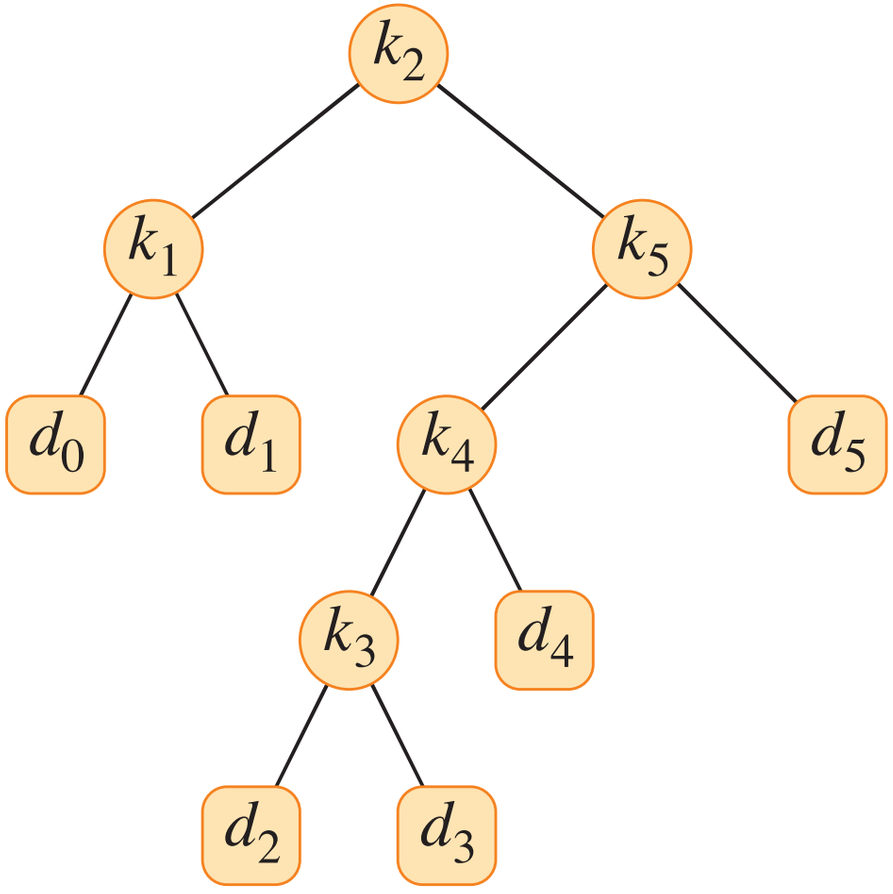
Binary Search Trees

- When contemplating the use of a binary search tree a repeated number of times, we must also consider whether some keys will be searched for more often than others.
- We may also want to represent frequent searches which return NIL.



Binary Search Trees

- When contemplating the use of a binary search tree a repeated number of times, we must also consider whether some keys will be searched for more often than others.
- We may also want to represent searches which return NIL.



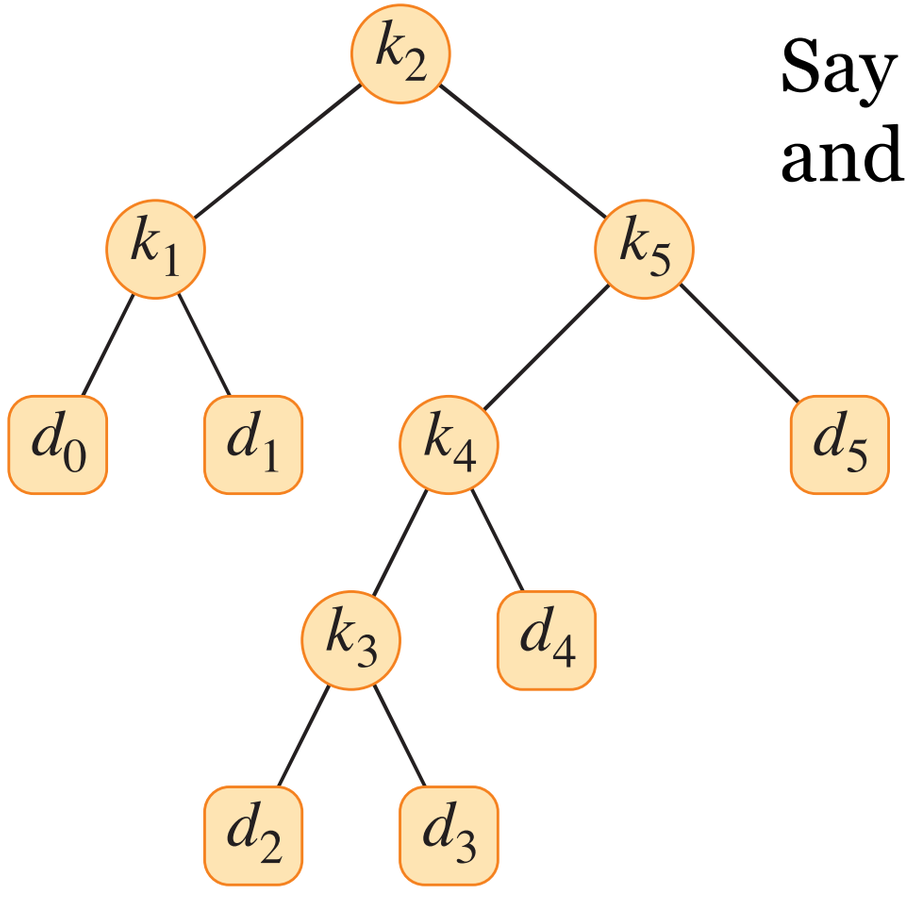
We represent those searches with dummy keys.

We can lump all searches for key values in between k_i and k_{i+1} into a dummy key d_i.

Thus dummy keys are always represented as leaves in the BST.

Binary Search Trees

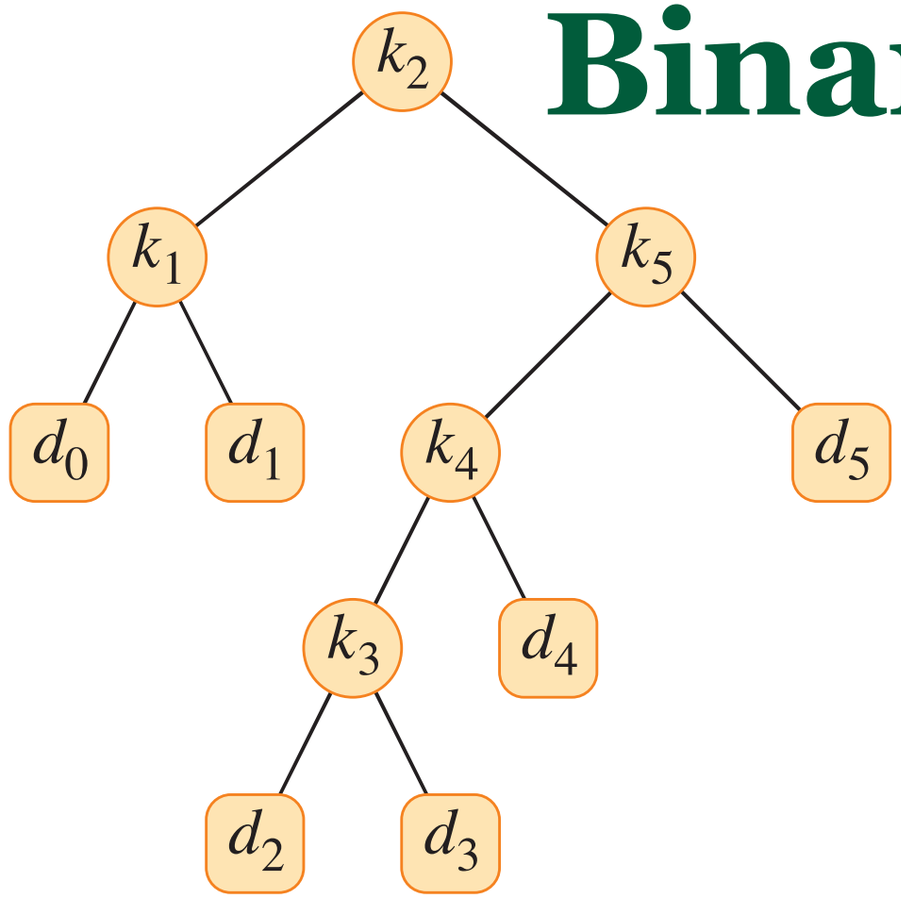
- When contemplating the use of a binary search tree a repeated number of times, we must also consider whether some keys will be searched for more often than others.
- We may also want to represent searches which return NIL.



Say the odds of searching for k_i are p_i, and the odds of searching for d_i, are q_i.

We can estimate the expected cost of searching in the BST as:

Binary Search Trees

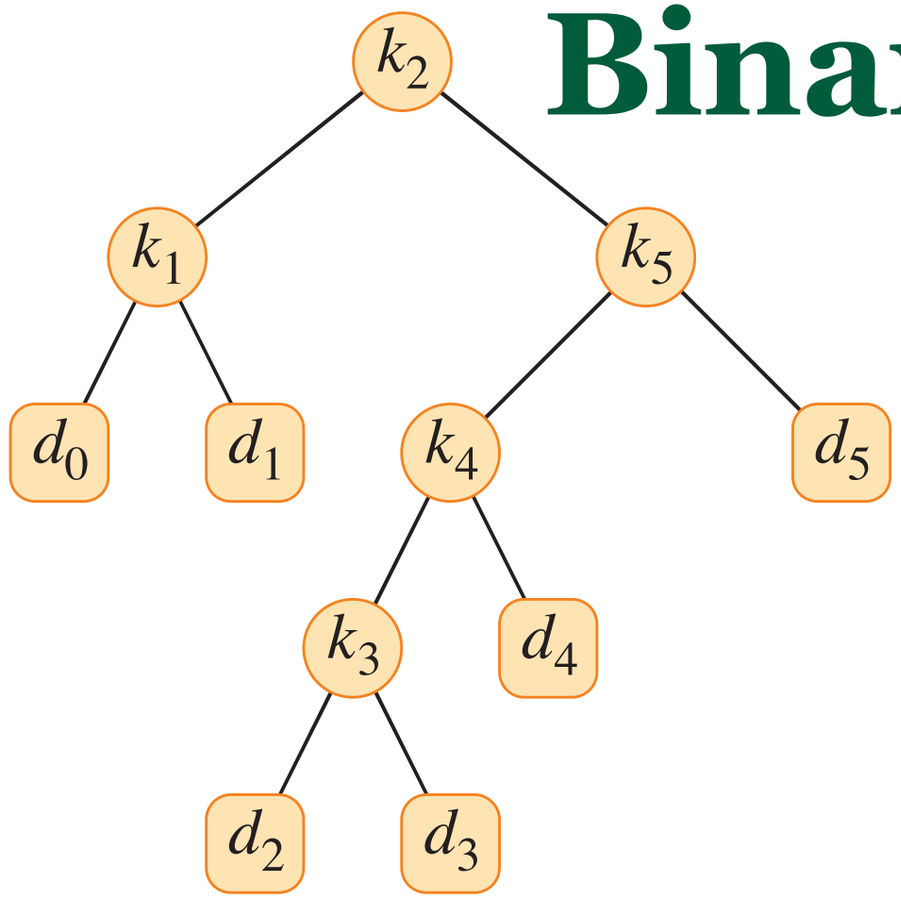


Say the odds of searching for k_i are p_i, and the odds of searching for d_i, are q_i.

We can estimate the expected cost of searching in the BST as:

$$\begin{aligned}\text{E}[\text{search cost in } T] &= \sum_{i=1}^{n}(\text{depth}_T(k_i)+1)\cdot p_i + \sum_{i=0}^{n}(\text{depth}_T(d_i)+1)\cdot q_i \\ &= 1 + \sum_{i=1}^{n}\text{depth}_T(k_i)\cdot p_i + \sum_{i=0}^{n}\text{depth}_T(d_i)\cdot q_i\end{aligned}$$

Binary Search Trees



Say the odds of searching for k_i are p_i, and the odds of searching for d_i, are q_i.

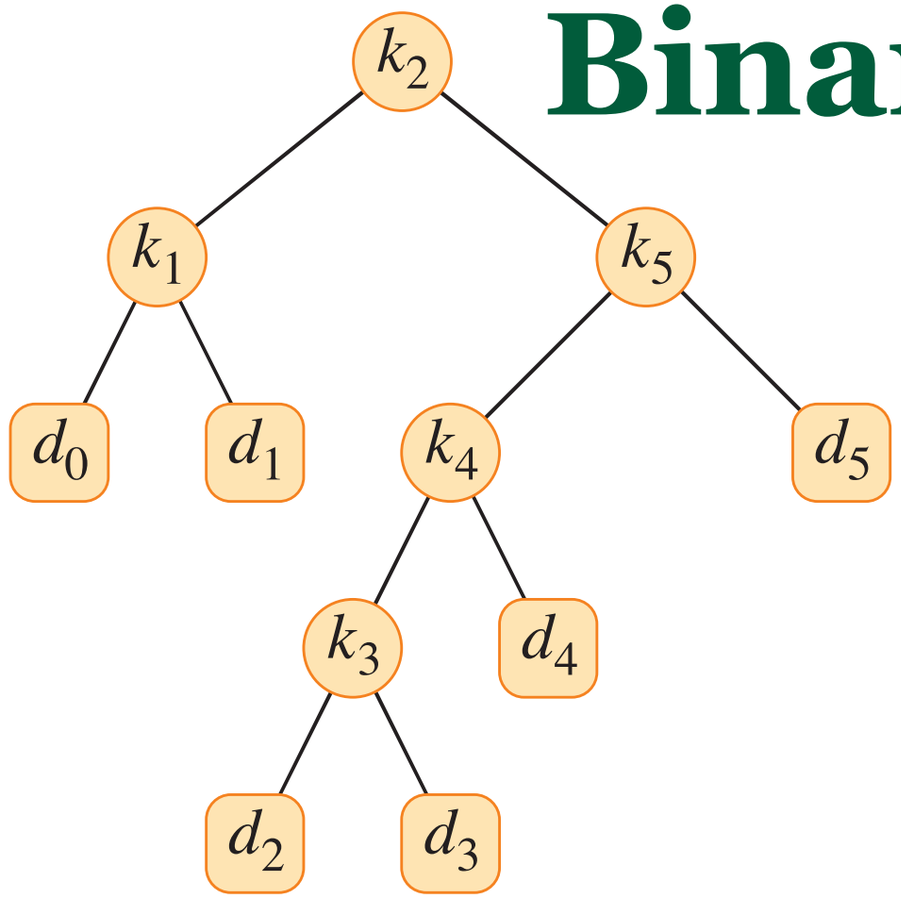
We can estimate the expected cost of searching in the BST as:

$$\text{E}[\text{search cost in } T] = \sum_{i=1}^{n} (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^{n} (\text{depth}_T(d_i) + 1) \cdot q_i$$

$$= 1 + \sum_{i=1}^{n} \text{depth}_T(k_i) \cdot p_i + \sum_{i=0}^{n} \text{depth}_T(d_i) \cdot q_i$$

$$\sum_{i=1}^{n} p_i + \sum_{i=0}^{n} q_i = 1 .$$

Binary Search Trees



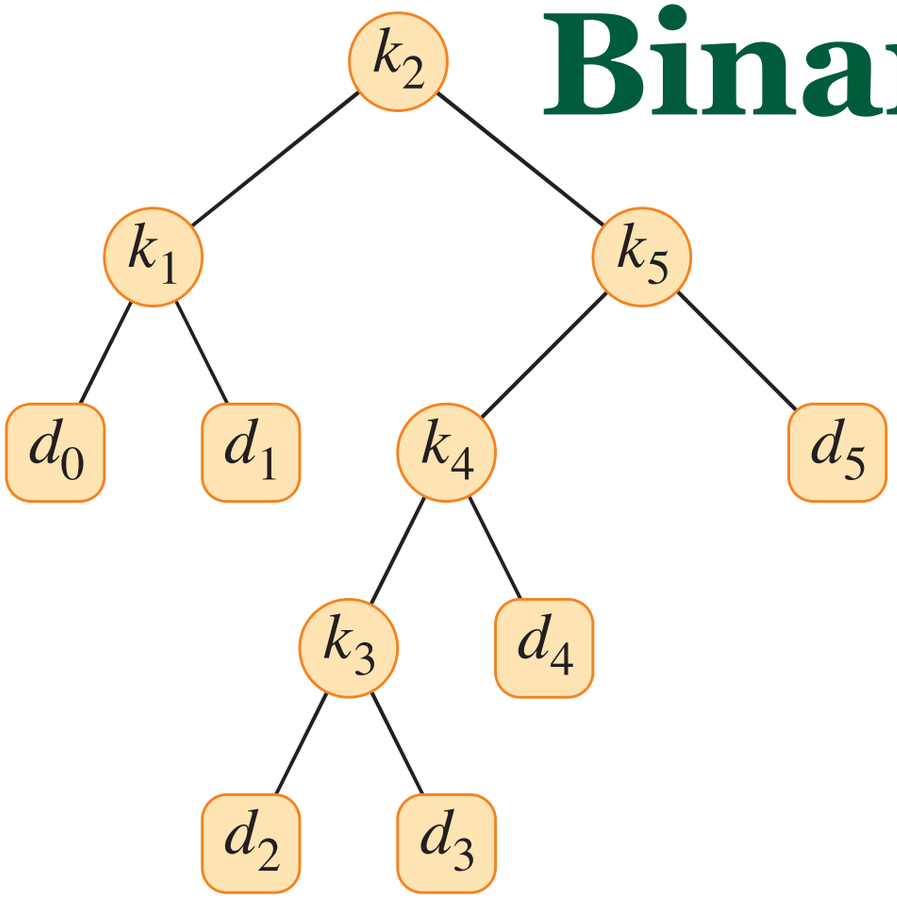
Say the odds of searching for k_i are p_i, and the odds of searching for d_i, are q_i.

We can estimate the expected cost of searching in the BST as:

$$\begin{aligned}\mathrm{E}[\text{search cost in } T] &= \sum_{i=1}^{n}(\mathrm{depth}_T(k_i)+1)\cdot p_i + \sum_{i=0}^{n}(\mathrm{depth}_T(d_i)+1)\cdot q_i \\ &= 1 + \sum_{i=1}^{n}\mathrm{depth}_T(k_i)\cdot p_i + \sum_{i=0}^{n}\mathrm{depth}_T(d_i)\cdot q_i\end{aligned}$$

Thus an optimal search tree, is not necessarily the tree with smallest height.

Binary Search Trees

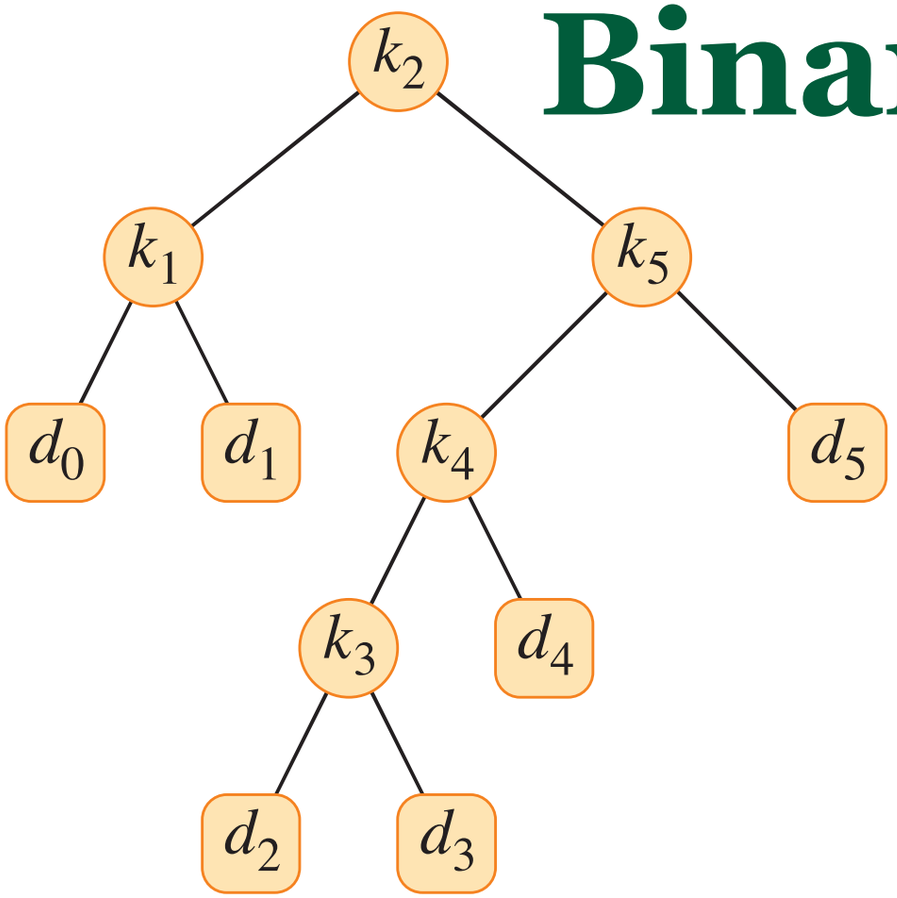


Say the odds of searching for k_i are p_i, and the odds of searching for d_i, are q_i.

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$\begin{aligned}\mathrm{E}\,[\text{search cost in } T] &= \sum_{i=1}^{n} (\mathrm{depth}_T(k_i) + 1) \cdot p_i \\ &\quad + \sum_{i=0}^{n} (\mathrm{depth}_T(d_i) + 1) \cdot q_i\end{aligned}$$

Binary Search Trees



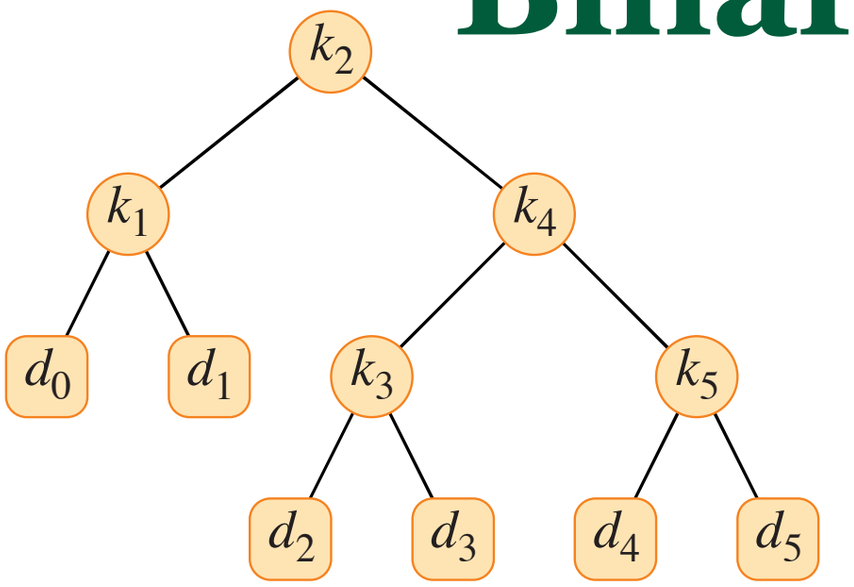
Say the odds of searching for k_i are p_i, and the odds of searching for d_i, are q_i.

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$\text{E}[\text{search cost in } T] = \sum_{i=1}^{n} (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^{n} (\text{depth}_T(d_i) + 1) \cdot q_i$$

node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	3	0.05	0.20
k_4	2	0.10	0.30
k_5	1	0.20	0.40
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	4	0.05	0.25
d_3	4	0.05	0.25
d_4	3	0.05	0.20
d_5	2	0.10	0.30
Total			2.75

Binary Search Trees



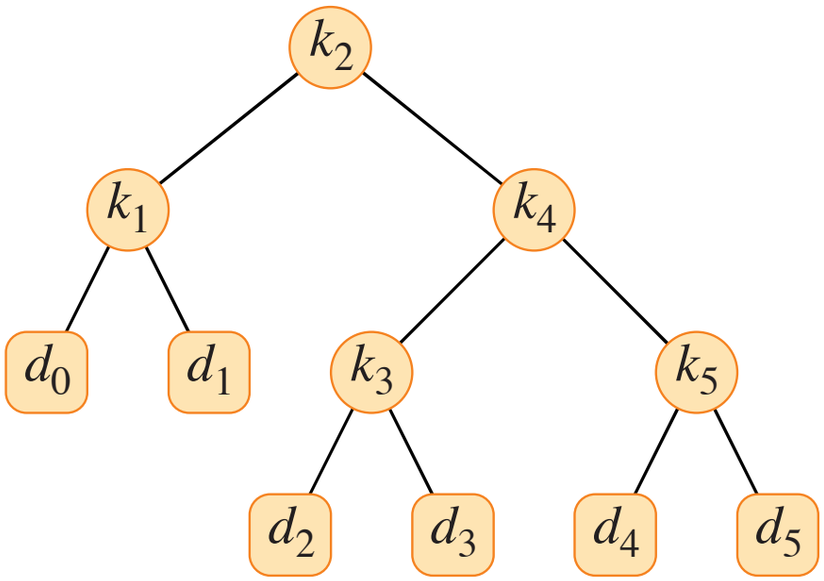
Say the odds of searching for k_i are p_i, and the odds of searching for d_i, are q_i.

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

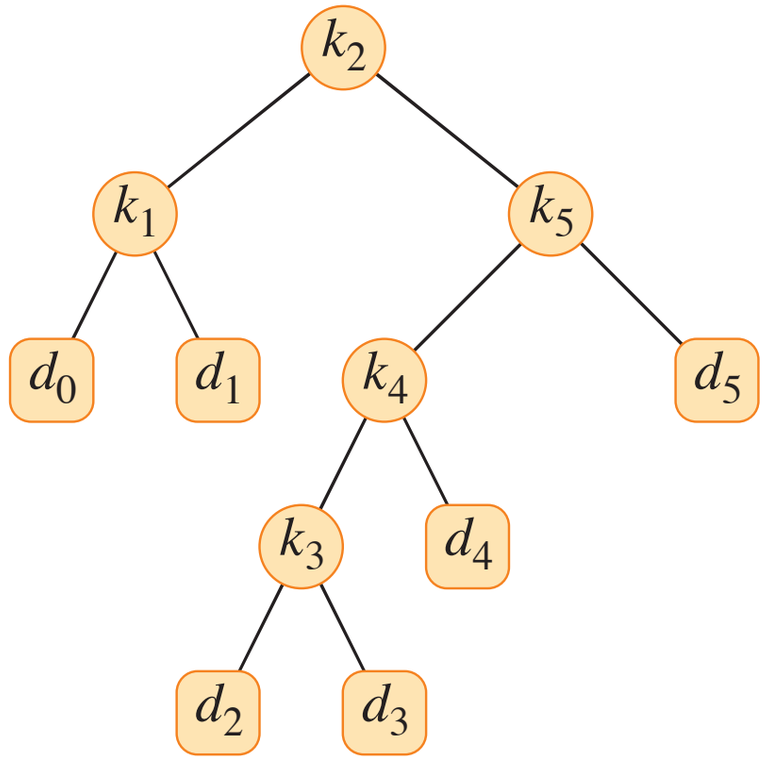
$$\text{E}[\text{search cost in } T] = \sum_{i=1}^{n} (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^{n} (\text{depth}_T(d_i) + 1) \cdot q_i$$

node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	2	0.05	0.15
k_4	1	0.10	0.20
k_5	2	0.20	0.60
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	3	0.05	0.20
d_3	3	0.05	0.20
d_4	3	0.05	0.20
d_5	3	0.10	0.40
Total			2.80

Binary Search Trees



node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	2	0.05	0.15
k_4	1	0.10	0.20
k_5	2	0.20	0.60
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	3	0.05	0.20
d_3	3	0.05	0.20
d_4	3	0.05	0.20
d_5	3	0.10	0.40
Total			2.80



node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	3	0.05	0.20
k_4	2	0.10	0.30
k_5	1	0.20	0.40
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	4	0.05	0.25
d_3	4	0.05	0.25
d_4	3	0.05	0.20
d_5	2	0.10	0.30
Total			2.75

Thus an optimal search tree, is not necessarily the tree with smallest height.

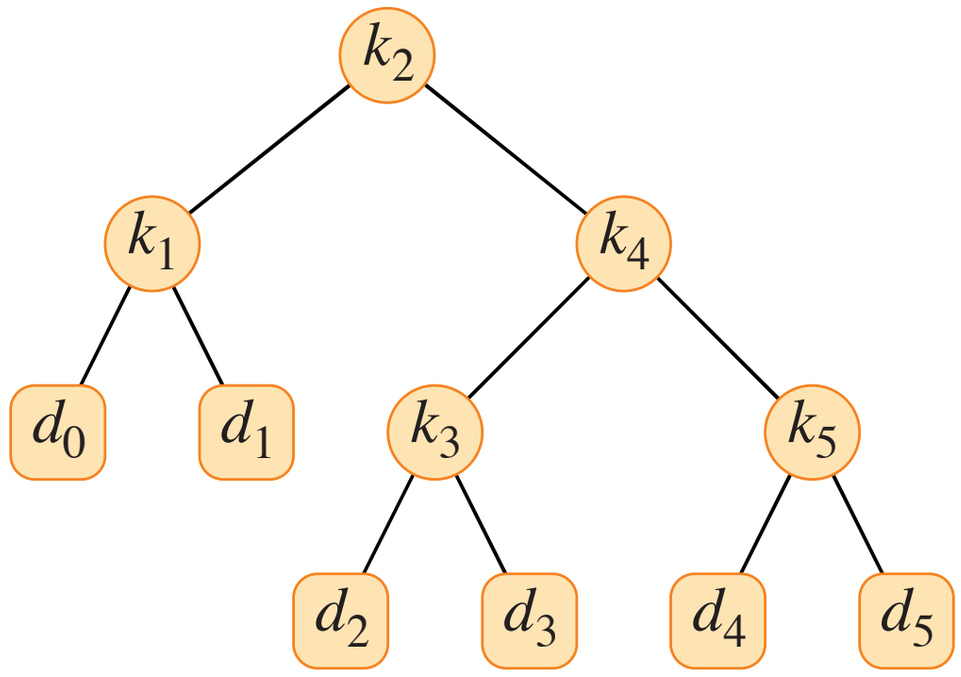
Optimal Binary Search Trees

- When contemplating the use of a binary search tree a repeated number of times, we must also consider whether some keys will be searched for more often than others.
- We showed an optimal search tree, is not necessarily the tree with smallest height. How can we construct an optimal BST?

- Let's begin by defining an optimal substructure. If an optimal tree contains a subtree, is it also optimal?

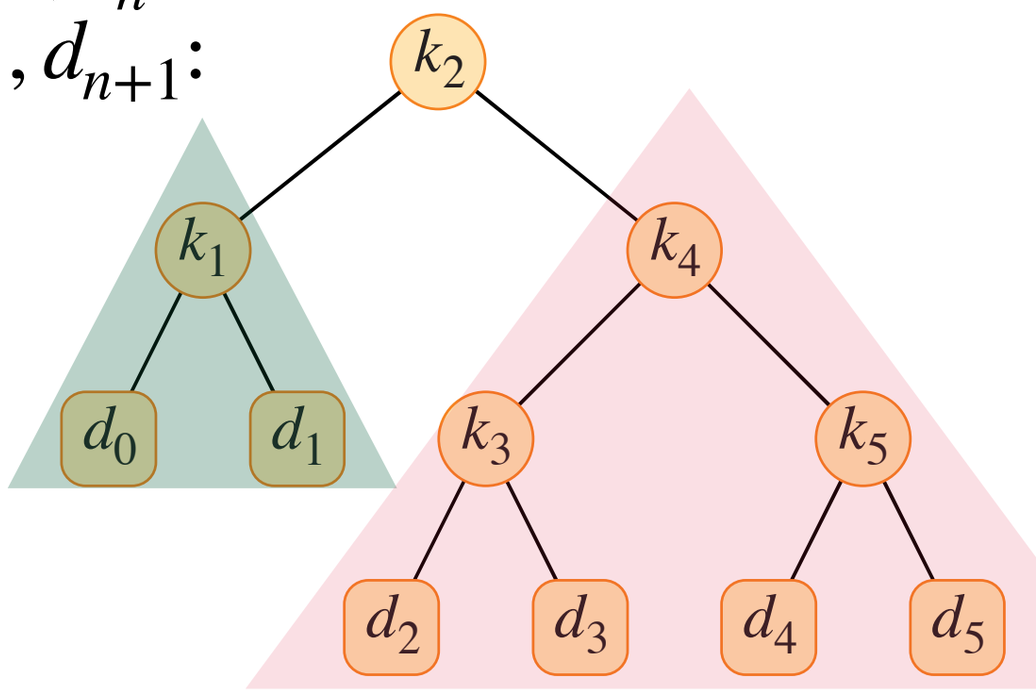
Optimal Binary Search Trees

- When contemplating the use of a binary search tree a repeated number of times, we must also consider whether some keys will be searched for more often than others.
- We showed an optimal search tree, is not necessarily the tree with smallest height. How can we construct an optimal BST?
- Let begin by defining an optimal substructure. If an optimal tree contains a subtree, is it also optimal?
- Yes!



Optimal Binary Search Trees

- When contemplating the use of a binary search tree a repeated number of times, we must also consider whether some keys will be searched for more often than others.
- We showed an optimal search tree, is not necessarily the tree with smallest height. How can we construct an optimal BST?
- Let begin by defining an optimal substructure. If an optimal tree contains a subtree, is it also optimal? Yes!
- To build an optimal tree with keys $k_1, \ldots, k_n$ and corresponding dummy keys, $d_0, \ldots, d_{n+1}$:
- Compare all trees with root k_r, for $r = 1,\ldots,n$, and optimal left and right subtrees.



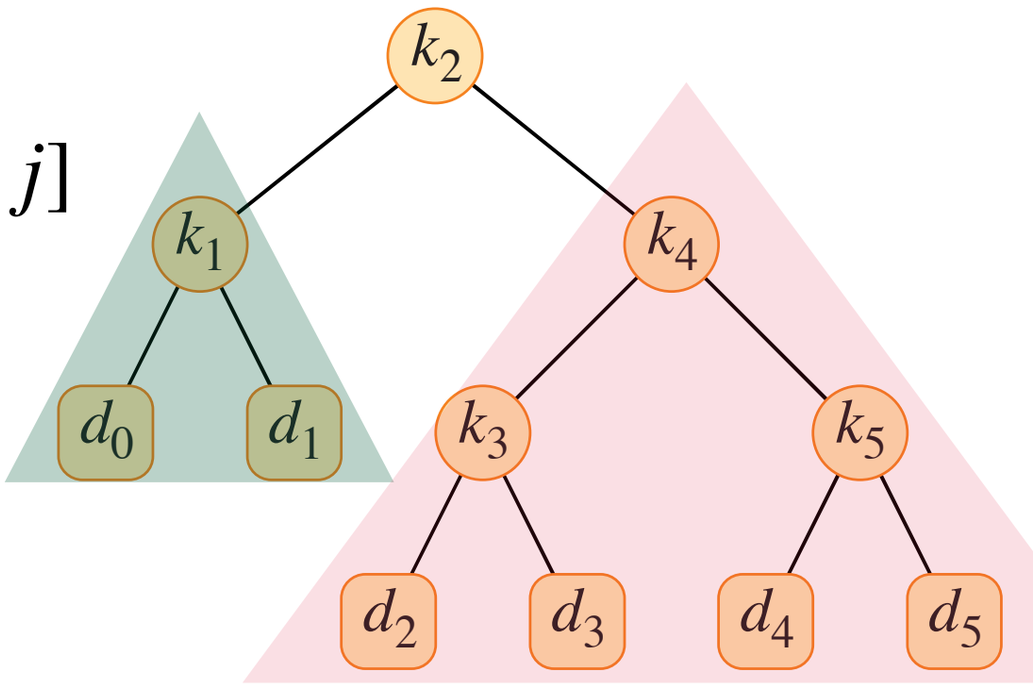
Optimal Binary Search Trees

- To build an optimal tree with keys $k_1, \ldots, k_n$ and corresponding dummy keys, $d_0, \ldots, d_{n+1}$:

  Compare all trees with root k_r, for $r = 1,\ldots,n$,
  and optimal left and right subtrees.

- If we know the left and right subtrees of k_r are optimal, how can we compute the expected cost of a search in a tree rooted at k_r?

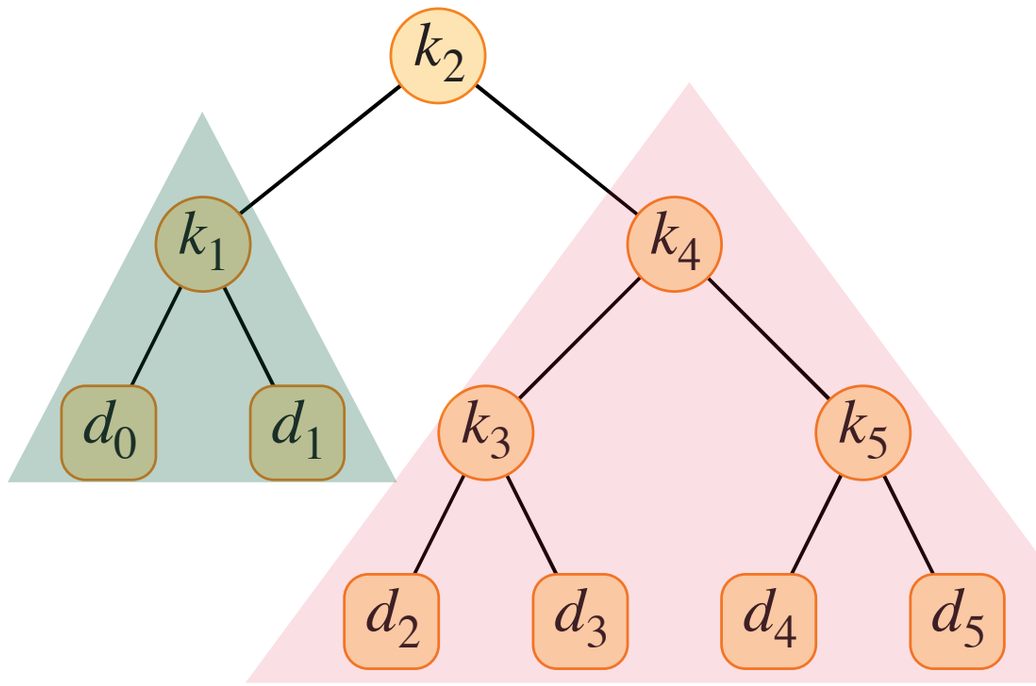
- Denote the expected cost of a search in optimal BST with keys $k_i, \ldots, k_j$, by: $e[i, j]$



Optimal Binary Search Trees

- If we know the left and right subtrees of k_r are optimal, how can we compute the expected cost of a search in a tree rooted at k_r?
- Denote the expected cost of a search in optimal BST with keys $k_i, \ldots, k_j$, by: $e[i, j]$. Say the tree is rooted at k_r.

$$e[i,j] = \sum_{l=i}^{j} (depth_{[i,j]}(k_l) + 1) \cdot p_l + \sum_{l=i-1}^{j} (depth_{[i,j]}(d_l) + 1) \cdot q_l$$



Optimal Binary Search Trees

- If we know the left and right subtrees of k_r are optimal, how can we compute the expected cost of a search in a tree rooted at k_r?
- Denote the expected cost of a search in optimal BST with keys $k_i, \ldots, k_j$, by: $e[i, j]$. Say the tree is rooted at k_r.

$$
\begin{aligned}
e[i,j] &= \sum_{l=i}^{j} (depth_{[i,j]}(k_l) + 1) \cdot p_l + \sum_{l=i-1}^{j} (depth_{[i,j]}(d_l) + 1) \cdot q_l \\
&= \sum_{l=i}^{r-1} (1 + depth_{[i,r-1]}(k_l) + 1) \cdot p_l + 1 \cdot p_r + \sum_{l=r+1}^{j} (1 + depth_{[r+1,j]}(k_l) + 1) \cdot p_l \\
&\quad + \sum_{l=i-1}^{r-1} (1 + depth_{[i,r-1]}(d_l) + 1) \cdot q_l + \sum_{l=r}^{j} (1 + depth_{[r,j]}(d_l) + 1) \cdot q_l
\end{aligned}
$$

Optimal Binary Search Trees

- If we know the left and right subtrees of k_r are optimal, how can we compute the expected cost of a search in a tree rooted at k_r?
- Denote the expected cost of a search in optimal BST with keys $k_i, \ldots, k_j$, by: $e[i, j]$. Say the tree is rooted at k_r.

$$
\begin{aligned}
e[i,j] &= \sum_{l=i}^{j} (depth_{[i,j]}(k_l) + 1) \cdot p_l + \sum_{l=i-1}^{j} (depth_{[i,j]}(d_l) + 1) \cdot q_l \\
&= \sum_{l=i}^{r-1} (1 + depth_{[i,r-1]}(k_l) + 1) \cdot p_l + 1 \cdot p_r + \sum_{l=r+1}^{j} (1 + depth_{[r+1,j]}(k_l) + 1) \cdot p_l \\
&\quad + \sum_{l=i-1}^{r-1} (1 + depth_{[i,r-1]}(d_l) + 1) \cdot q_l + \sum_{l=r}^{j} (1 + depth_{[r,j]}(d_l) + 1) \cdot q_l \\
&= e[i, r-1] + e[r+1, j] + p_r + \sum_{l=i}^{r-1} p_l + \sum_{l=r+1}^{j} p_l + \sum_{l=i-1}^{r-1} q_l + \sum_{l=r}^{j} q_l
\end{aligned}
$$

Optimal Binary Search Trees

- If we know the left and right subtrees of k_r are optimal, how can we compute the expected cost of a search in a tree rooted at k_r?
- Denote the expected cost of a search in optimal BST with keys $k_i, \ldots, k_j$, by: $e[i, j]$

$$
\begin{aligned}
e[i,j] &= \sum_{l=i}^{j} (depth_{[i,j]}(k_l) + 1) \cdot p_l + \sum_{l=i-1}^{j} (depth_{[i,j]}(d_l) + 1) \cdot q_l \\
&= \sum_{l=i}^{r-1} (1 + depth_{[i,r-1]}(k_l) + 1) \cdot p_l + 1 \cdot p_r + \sum_{l=r+1}^{j} (1 + depth_{[r+1,j]}(k_l) + 1) \cdot p_l \\
&\quad + \sum_{l=i-1}^{r-1} (1 + depth_{[i,r-1]}(d_l) + 1) \cdot q_l + \sum_{l=r}^{j} (1 + depth_{[r,j]}(d_l) + 1) \cdot q_l \\
&= e[i, r-1] + e[r+1, j] + \sum_{l=i}^{j} p_l + \sum_{l=i-1}^{j} q_l
\end{aligned}
$$

Optimal Binary Search Trees

- If we know the left and right subtrees of k_r are optimal, how can we compute the expected cost of a search in a tree rooted at k_r?
- Denote the expected cost of a search in optimal BST with keys $k_i, \ldots, k_j$, by: $e[i, j]$

$$
\begin{aligned}
e[i,j] &= \sum_{l=i}^{j} (depth_{[i,j]}(k_l) + 1) \cdot p_l + \sum_{l=i-1}^{j} (depth_{[i,j]}(d_l) + 1) \cdot q_l \\
&= \sum_{l=i}^{r-1} (1 + depth_{[i,r-1]}(k_l) + 1) \cdot p_l + 1 \cdot p_r + \sum_{l=r+1}^{j} (1 + depth_{[r+1,j]}(k_l) + 1) \cdot p_l \\
&\quad + \sum_{l=i-1}^{r-1} (1 + depth_{[i,r-1]}(d_l) + 1) \cdot q_l + \sum_{l=r}^{j} (1 + depth_{[r,j]}(d_l) + 1) \cdot q_l \\
&= e[i, r-1] + e[r+1, j] + w[i, j]
\end{aligned}
$$

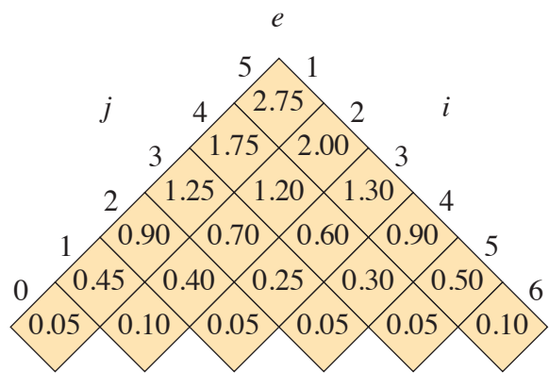
Optimal Binary Search Trees

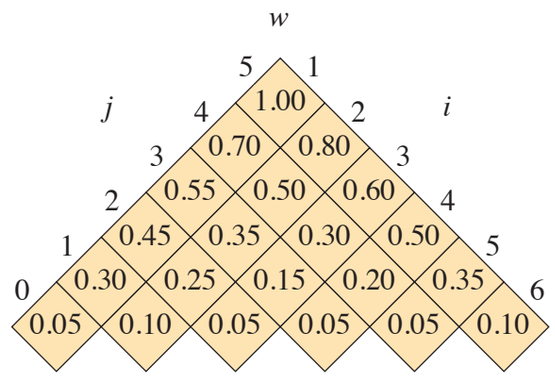
```
OPTIMAL-BST(p, q, n)
 1  let e[1 : n + 1, 0 : n], w[1 : n + 1, 0 : n],
          and root[1 : n, 1 : n] be new tables
 2  for i = 1 to n + 1
 3      e[i, i − 1] = q_{i−1}
 4      w[i, i − 1] = q_{i−1}
 5  for l = 1 to n
 6      for i = 1 to n − l + 1
 7          j = i + l − 1
 8          e[i, j] = ∞
 9          w[i, j] = w[i, j − 1] + p_j + q_j
10          for r = i to j
11              t = e[i, r − 1] + e[r + 1, j] + w[i, j]
12              if t < e[i, j]
13                  e[i, j] = t
14                  root[i, j] = r
15  return e and root
```

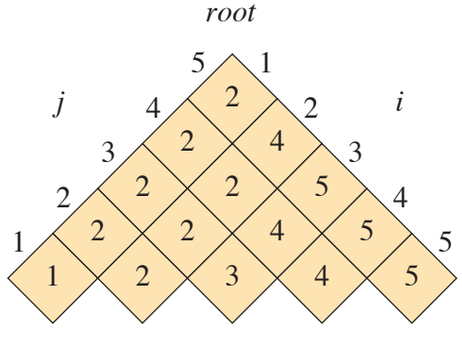
Optimal Binary Search Trees

```
OPTIMAL-BST(p, q, n)
 1  let e[1 : n + 1, 0 : n], w[1 : n + 1, 0 : n],
          and root[1 : n, 1 : n] be new tables
 2  for i = 1 to n + 1
 3      e[i, i − 1] = q_{i−1}
 4      w[i, i − 1] = q_{i−1}
 5  for l = 1 to n
 6      for i = 1 to n − l + 1
 7          j = i + l − 1
 8          e[i, j] = ∞
 9          w[i, j] = w[i, j − 1] + p_j + q_j
10          for r = i to j
11              t = e[i, r − 1] + e[r + 1, j] + w[i, j]
12              if t < e[i, j]
13                  e[i, j] = t
14                  root[i, j] = r
15  return e and root
```

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



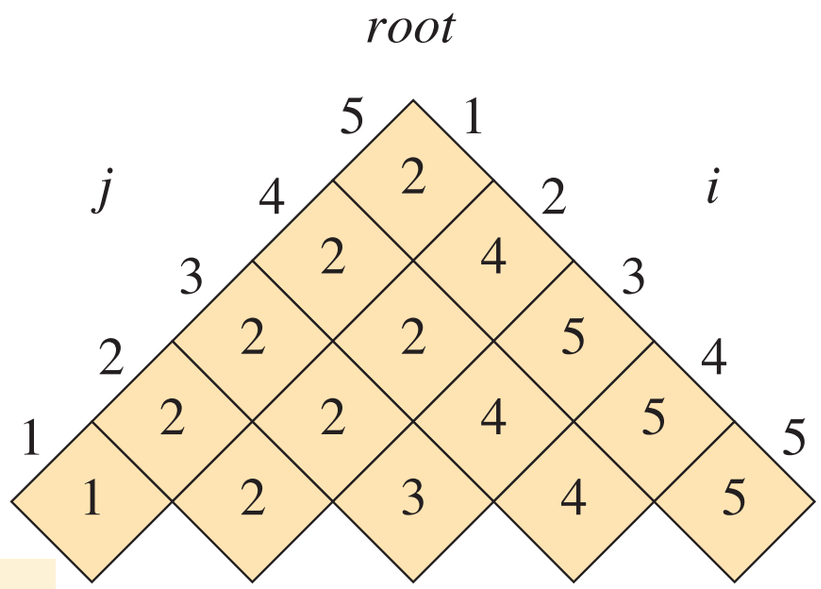


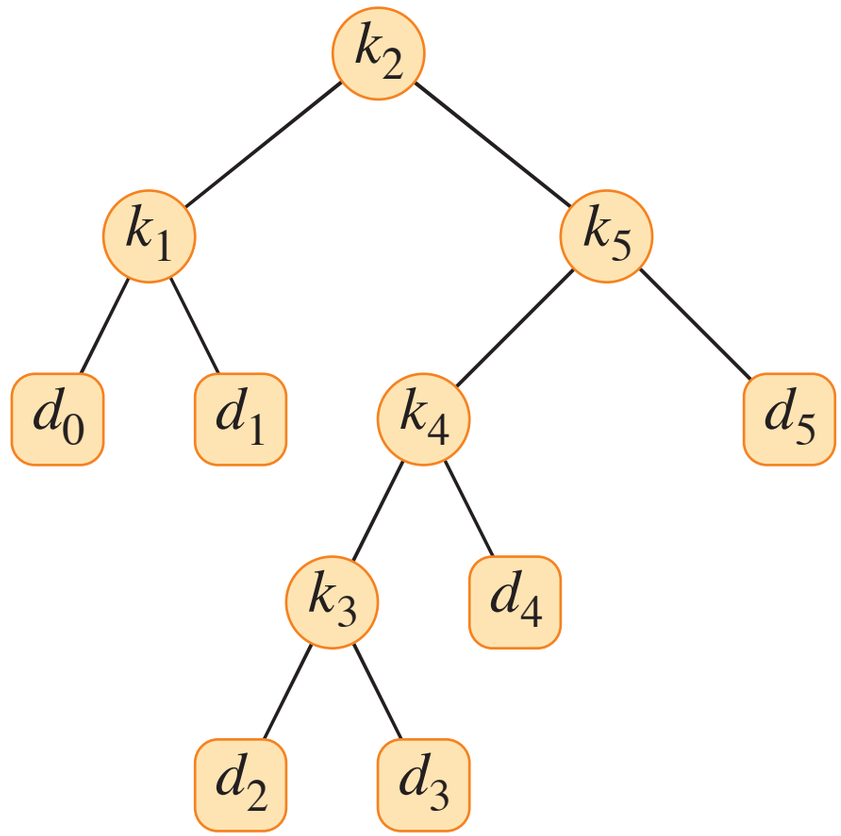


Optimal Binary Search Trees

```
CONSTRUCT-OPTIMAL-BST(root, n)
    print "Optimal BST structure:"
    PRINT-TREE(root, 1, n, "root")
```

```
PRINT-TREE(root, i, j, position)
    if i > j
        return
    r = root[i, j]
    print "k_" + r + " is the " + position
    PRINT-TREE(root, i, r - 1, "left child of k_" + r)
    PRINT-TREE(root, r + 1, j, "right child of k_" + r)
```

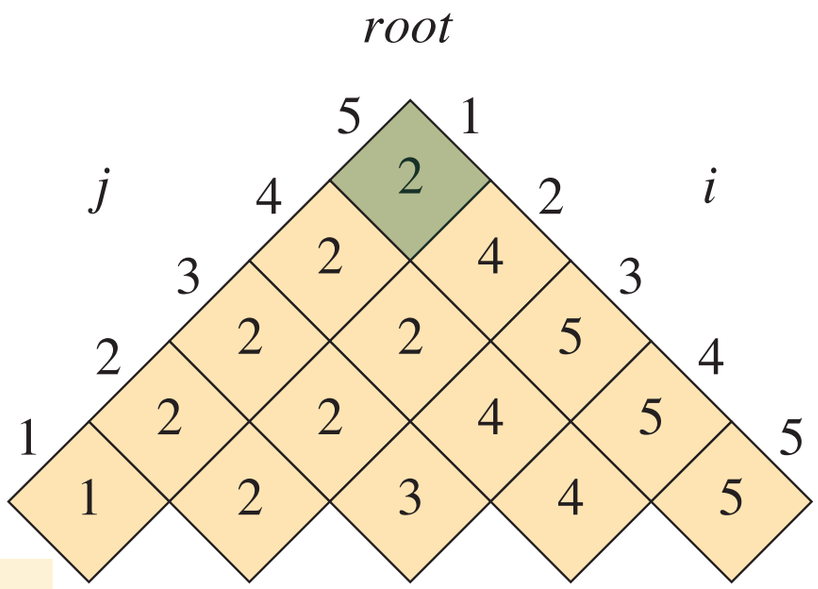


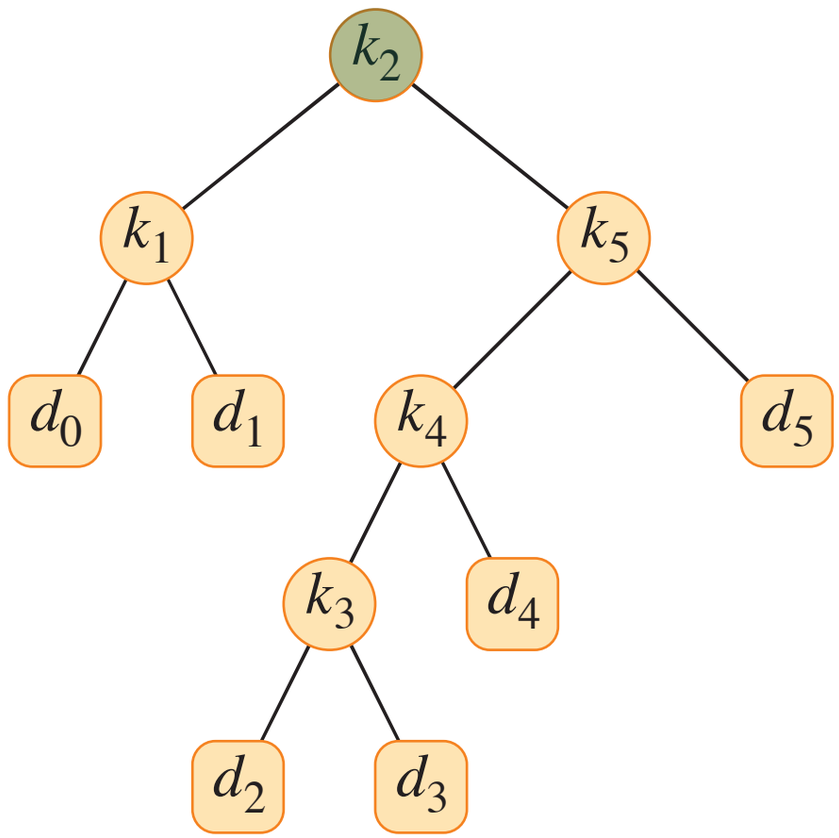


Optimal Binary Search Trees

```
CONSTRUCT-OPTIMAL-BST(root, n)
    print "Optimal BST structure:"
    PRINT-TREE(root, 1, n, "root")
```

```
PRINT-TREE(root, i, j, position)
    if i > j
        return
    r = root[i, j]
    print "k_" + r + " is the " + position
    PRINT-TREE(root, i, r - 1, "left child of k_" + r)
    PRINT-TREE(root, r + 1, j, "right child of k_" + r)
```

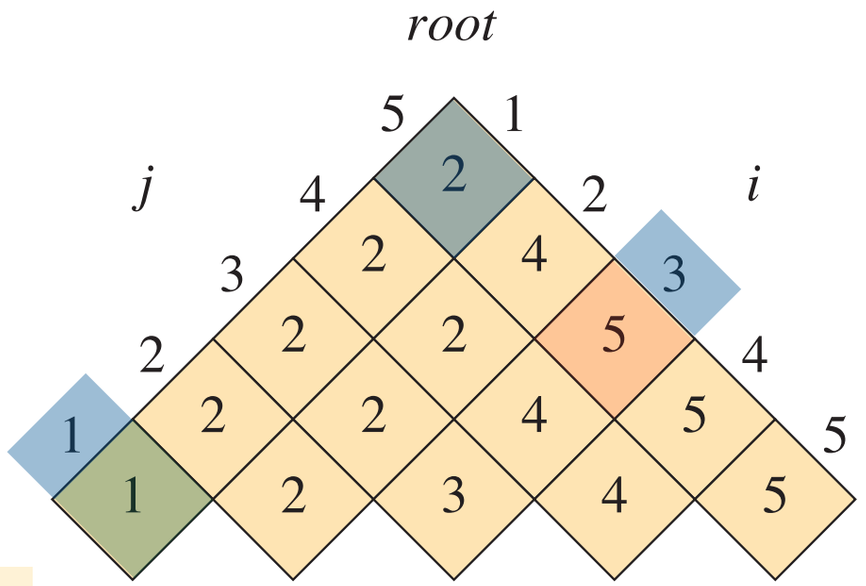


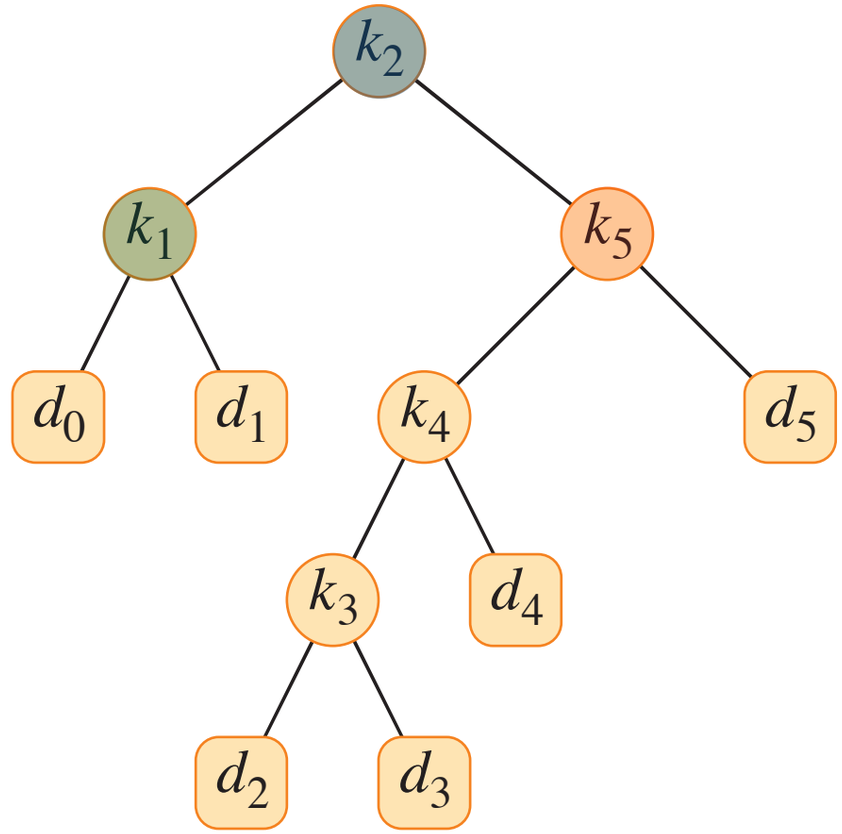


Optimal Binary Search Trees

```
CONSTRUCT-OPTIMAL-BST(root, n)
    print "Optimal BST structure:"
    PRINT-TREE(root, 1, n, "root")
```

```
PRINT-TREE(root, i, j, position)
    if i > j
        return
    r = root[i, j]
    print "k_" + r + " is the " + position
    PRINT-TREE(root, i, r - 1, "left child of k_" + r)
    PRINT-TREE(root, r + 1, j, "right child of k_" + r)
```

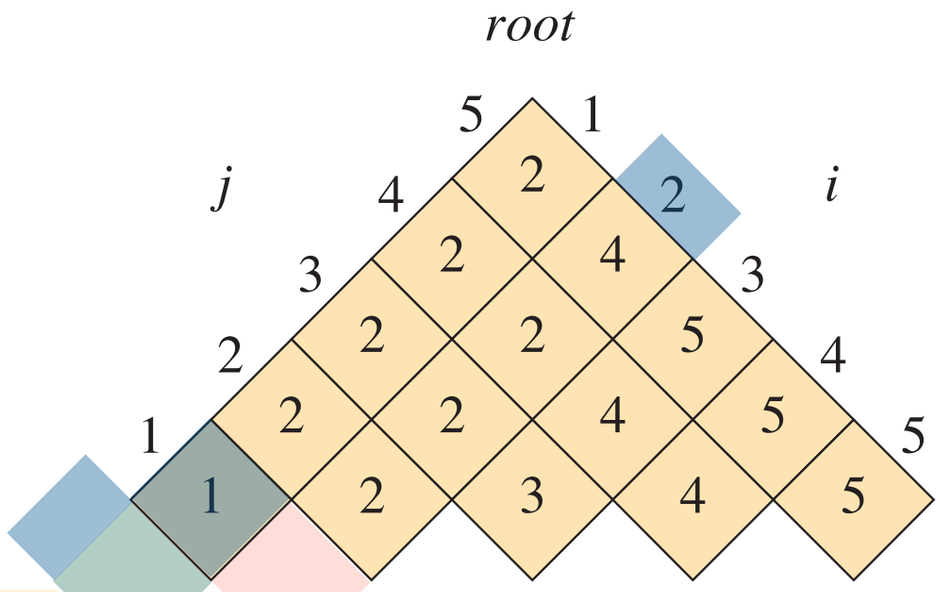


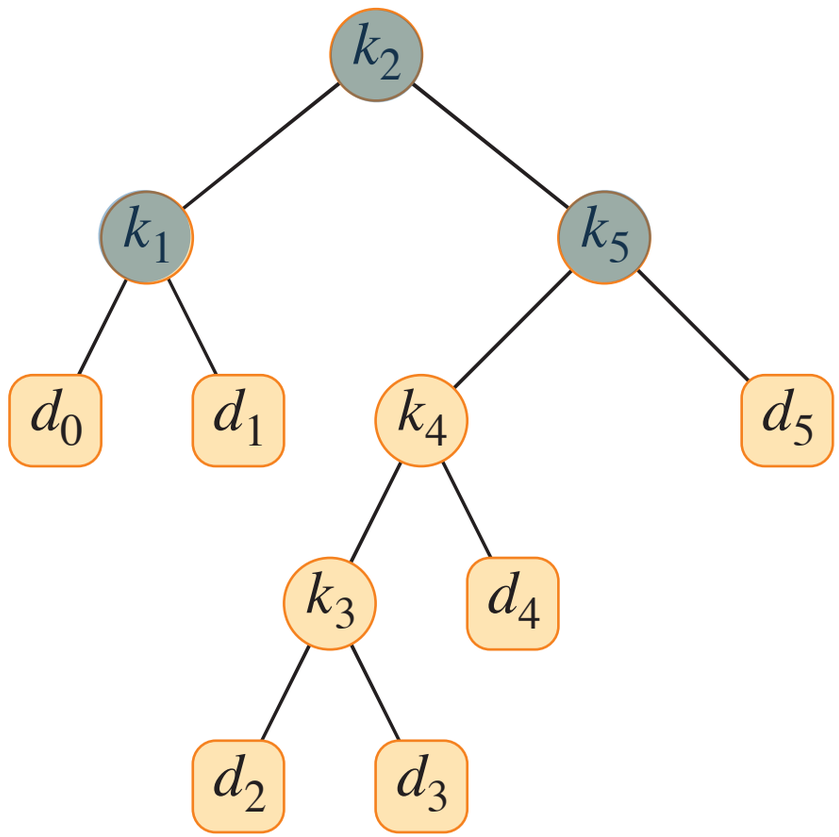


Optimal Binary Search Trees

```
CONSTRUCT-OPTIMAL-BST(root, n)
    print "Optimal BST structure:"
    PRINT-TREE(root, 1, n, "root")
```

```
PRINT-TREE(root, i, j, position)
    if i > j
        return
    r = root[i, j]
    print "k_" + r + " is the " + position
    PRINT-TREE(root, i, r - 1, "left child of k_" + r)
    PRINT-TREE(root, r + 1, j, "right child of k_" + r)
```

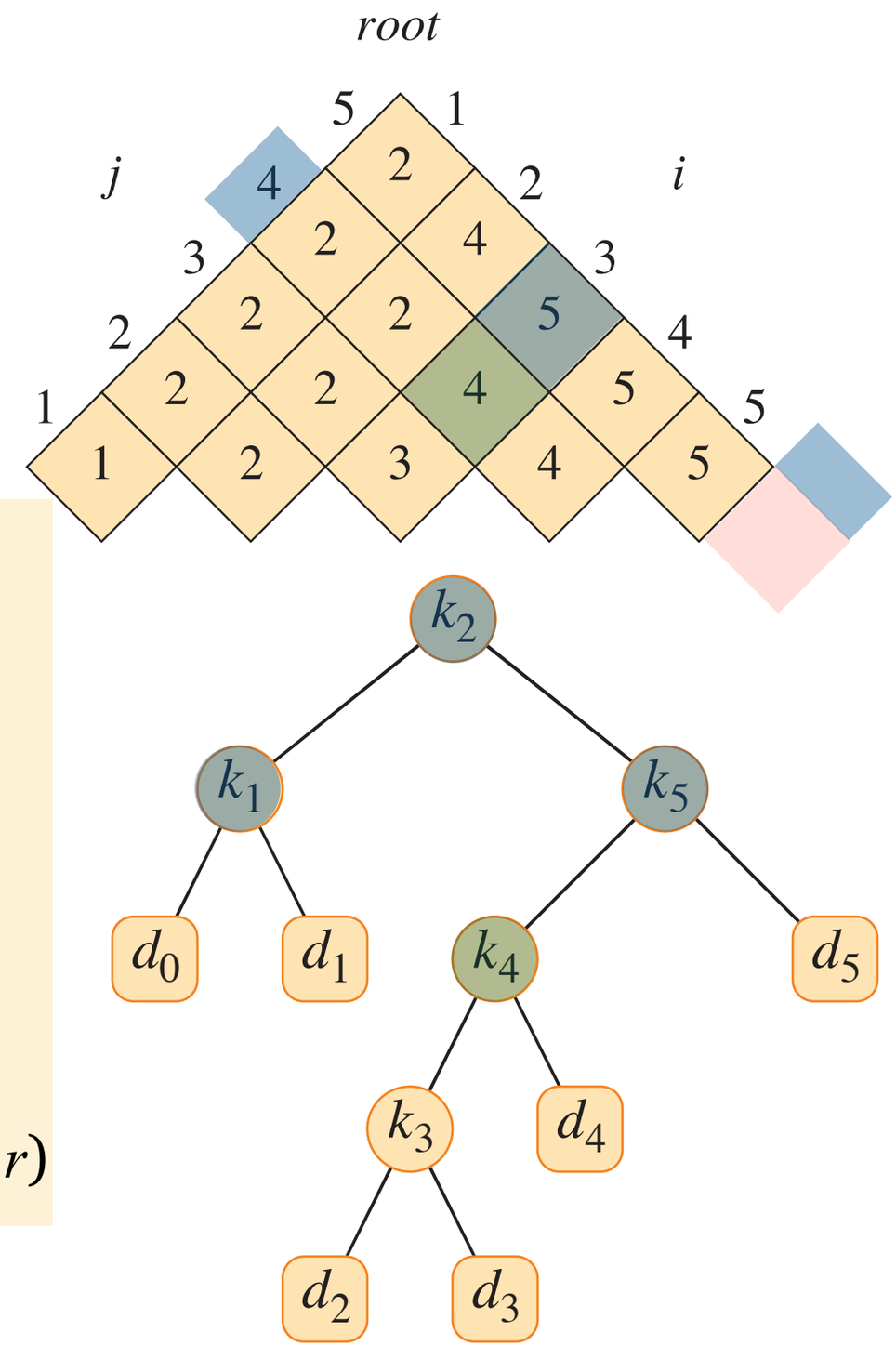




Optimal Binary Search Trees

```
CONSTRUCT-OPTIMAL-BST(root, n)
    print "Optimal BST structure:"
    PRINT-TREE(root, 1, n, "root")
```

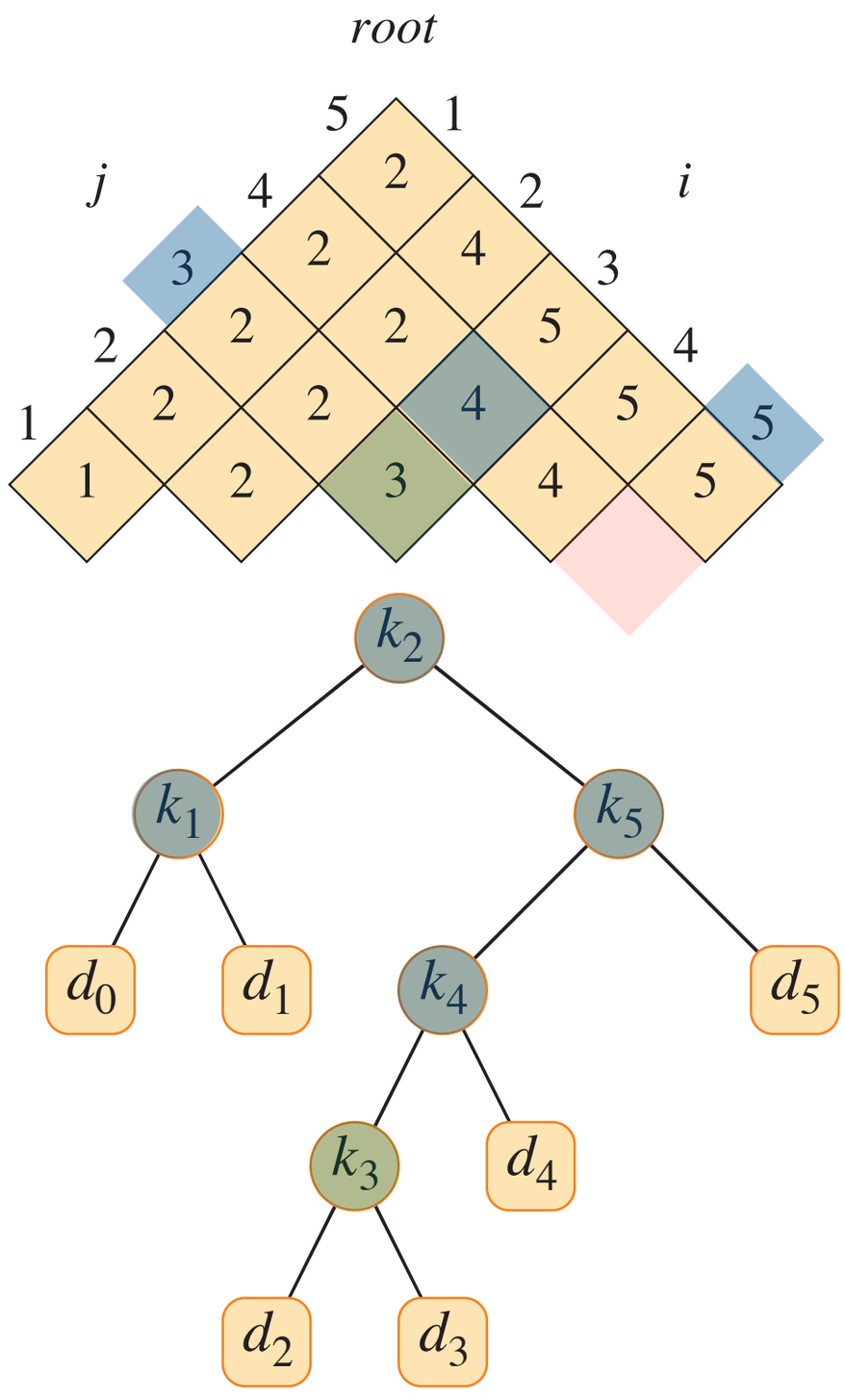
```
PRINT-TREE(root, i, j, position)
    if i > j
        return
    r = root[i, j]
    print "k_" + r + " is the " + position
    PRINT-TREE(root, i, r - 1, "left child of k_" + r)
    PRINT-TREE(root, r + 1, j, "right child of k_" + r)
```



Optimal Binary Search Trees

```
CONSTRUCT-OPTIMAL-BST(root, n)
    print "Optimal BST structure:"
    PRINT-TREE(root, 1, n, "root")
```

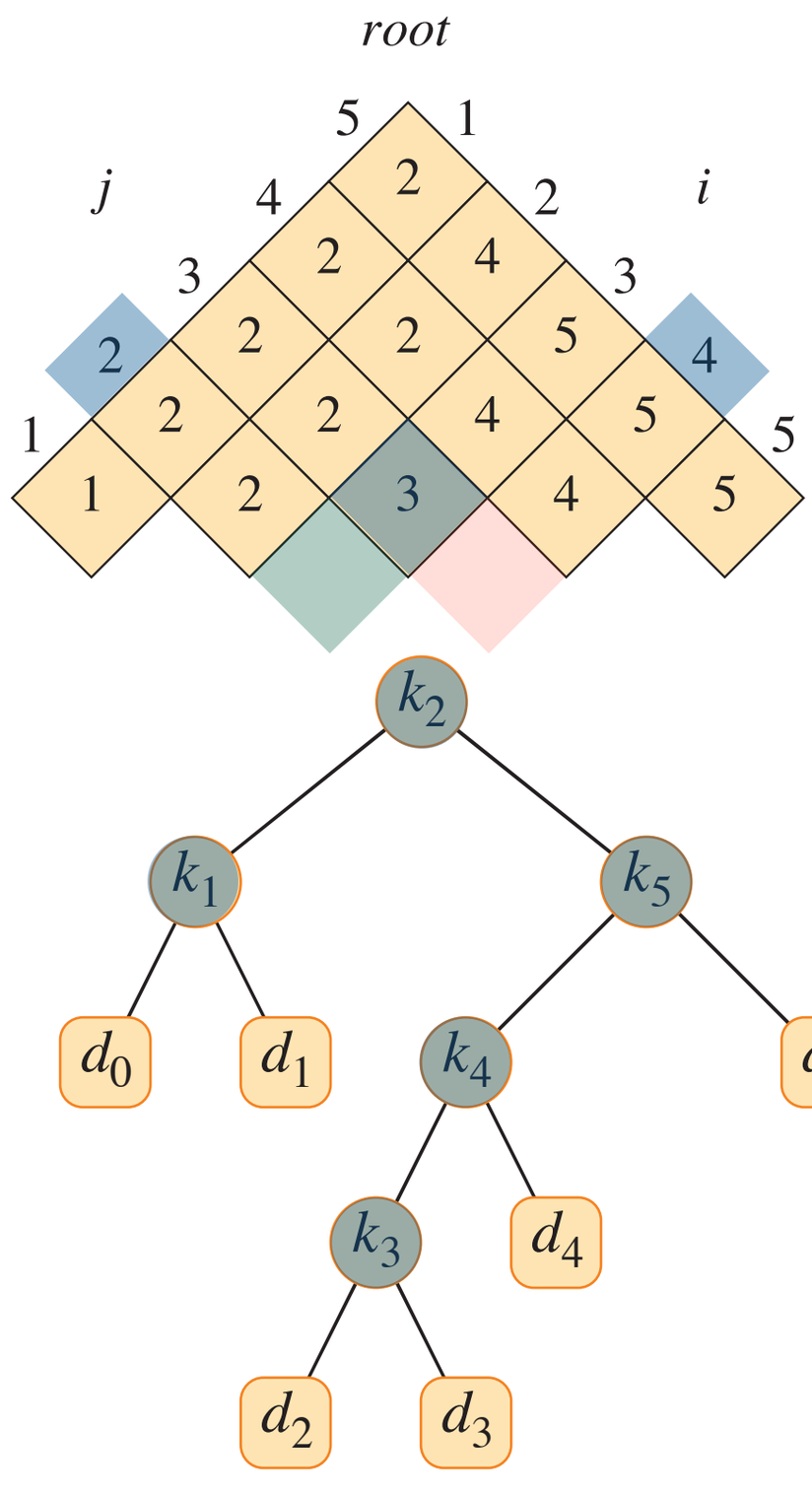
```
PRINT-TREE(root, i, j, position)
if i > j
    return
r = root[i, j]
print "k_" + r + " is the " + position
PRINT-TREE(root, i, r - 1, "left child of k_" + r)
PRINT-TREE(root, r + 1, j, "right child of k_" + r)
```



Optimal Binary Search Trees

```
CONSTRUCT-OPTIMAL-BST(root, n)
    print "Optimal BST structure:"
    PRINT-TREE(root, 1, n, "root")

PRINT-TREE(root, i, j, position)
    if i > j
        return
    r = root[i, j]
    print "k_" + r + " is the " + position
    PRINT-TREE(root, i, r - 1, "left child of k_" + r)
    PRINT-TREE(root, r + 1, j, "right child of k_" + r)
```

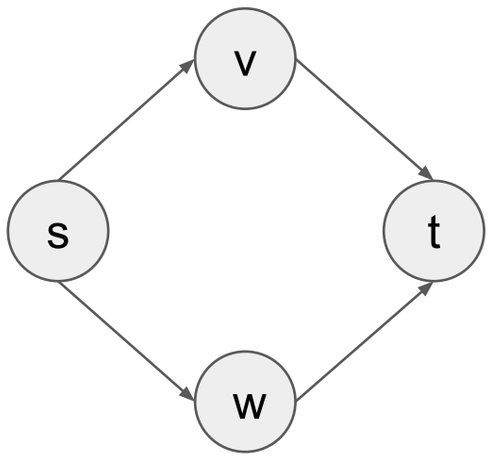


Exercise

- You are given a directed acyclic graph with real-valued edge weights and two distinguished vertices s and t. Describe a dynamic-programming approach for finding a longest weighted simple path from s to t.
- What is the running time of your algorithm?

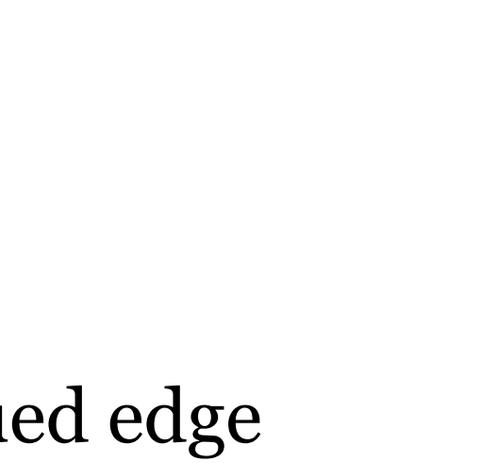
Exercise

- You are given a directed acyclic graph with real-valued edge weights and two distinguished vertices s and t. Describe a dynamic-programming approach for finding a longest weighted simple path from s to t.
- What is the running time of your algorithm?



Exercise

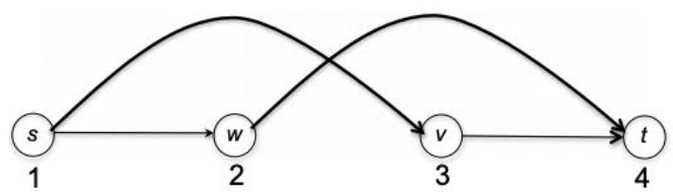
- You are given a directed acyclic graph with real-valued edge weights and two distinguished vertices s and t. Describe a dynamic-programming approach for finding a longest weighted simple path from s to t.
- What is the running time of your algorithm?



Topological Orderings

Let $G = (V, E)$ be a directed graph. A *topological ordering* of G is an assignment $f(v)$ of every vertex $v \in V$ to a different number such that:

for every $(v, w) \in E$, $f(v) < f(w)$.



Exercise

```
LONGEST-PATH-DAG(G, s, t)
    Perform topological sort on G to get ordering v1, v2, ..., vn
    Let d[v] = length of longest path from s to v
    Let π[v] = predecessor of v on longest path from s
    for each vertex v in G
        d[v] = -∞
        π[v] = NIL
    d[s] = 0
    for each vertex u in topological order
        if d[u] ≠ -∞
            for each edge (u, v) with weight w(u, v)
                if d[u] + w(u, v) > d[v]
                    d[v] = d[u] + w(u, v)
                    π[v] = u
    if d[t] = -∞
        print "No path from s to t"
    else
        print "Longest path has weight " + d[t]
        PRINT-PATH(π, s, t)
```

```
PRINT-PATH(π, s, t)
    if t = s
        print s
    else if π[t] = NIL
        print "No path exists"
    else
        PRINT-PATH(π, s, π[t])
        print t
```

Exercise

```
LONGEST-PATH-DAG(G, s, t)
    Perform topological sort on G to get ordering v1, v2, ..., vn
    Let d[v] = length of longest path from s to v
    Let π[v] = predecessor of v on longest path from s
    for each vertex v in G
        d[v] = -∞
        π[v] = NIL
    d[s] = 0
    for each vertex u in topological order
        if d[u] ≠ -∞
            for each edge (u, v) with weight w(u, v)
                if d[u] + w(u, v) > d[v]
                    d[v] = d[u] + w(u, v)
                    π[v] = u
    if d[t] = -∞
        print "No path from s to t"
    else
        print "Longest path has weight " + d[t]
        PRINT-PATH(π, s, t)
```

Topological ordering ensures that when we process vertex v, we've already computed optimal solutions for all vertices that can reach v.

```
PRINT-PATH(π, s, t)
    if t = s
        print s
    else if π[t] = NIL
        print "No path exists"
    else
        PRINT-PATH(π, s, π[t])
        print t
```